Generalized Batched Decomposition Key-Switching for CKKS

Antonio J. Peña, Zaira Pindado, and Hugo Sanz-González

Barcelona Supercomputing Center (BSC)
{antonio.pena, zaira.pindado, hugo.sanzgonzalez}@bsc.es

Abstract. A basic property of lattice-based homomorphic encryption schemes is that ciphertexts carry an inherent noise that accumulates as computation advances. The key-switching procedure of these schemes allows the homomorphic re-encryption of a ciphertext under a new secret key, but it is a major source of noise growth. Several strategies have been proposed to reduce the amount of noise introduced by key-switching in the CKKS scheme. In this work, we generalize the RNS-based decomposition technique to an arbitrary number of input polynomials and secret keys. Performing multiple relinearizations and rotations in batch enables one to hoist the modulus lowering and decomposition steps of key-switching, respectively, which restricts noise growth and avoids redundant computation. We prove the security and correctness of all the proposed algorithms, and derive explicit noise bounds for them. Practical examples illustrate the impact of these optimizations on applications like private deep neural network inference. In addition, we provide—to the best of our knowledge—the first noise analysis of the double-hoisted BSGS matrix-vector multiplication algorithm of Bossuat *et al.* (2021), which is a building block of state-of-the-art CKKS bootstrapping circuits. Our analysis additionally reveals an optimization concerning the automorphism applications during its giant steps.

Keywords: Fully Homomorphic Encryption · Key-Switching · Gadget Decomposition

1 Introduction

Fully Homomorphic Encryption (FHE) [17] is a cryptographic primitive that enables direct computation on encrypted data without the need for intermediate decryptions. The encrypted result is approximately the same as if it had been produced by the traditional method (decryption, computation, and encryption) and may be decrypted regularly using the secret key. FHE is being widely explored in the context of privacy-preserving computations thanks to the appearance of a number of open-source software libraries [3, 15, 34]. The rapid development and interest in FHE has turned this cryptosystem into an attractive option to provide privacy guarantees in a variety of scenarios, such as deep neural network (DNN) inference [30].

FHE schemes that are secure under the Learning with Errors (LWE) [27, 31] assumption or its ring variant (RLWE) [33] contain an inherent error that grows with homomorphic computations; this error growth needs to be maintained under control to guarantee a correct decryption. A large portion of research in the field has focused both on reducing the noise growth and on improving the computational efficiency of homomorphic operations. Key-switching converts a ciphertext under a secret key into an equivalent ciphertext under another secret key. This re-encryption procedure involves a public evaluation key, which is derived from the source and target secret keys by the key generator. Many homomorphic operations require key-switching in order to cancel the effect of encrypted arithmetic on the decryption circuit. Unfortunately, this operation represents the main bottleneck in the performance of nonlinear operations such as multiplication and slot rotation.

Many approaches have been proposed in the literature to improve the performance and to reduce noise in lattice-based FHE schemes. Residue number systems (RNS) improve computational efficiency by representing large integers modulo Q as tuples of residues modulo smaller coprime integers $q_0, \ldots, q_L$ whose product is Q. A full-RNS variant of the CKKS scheme was introduced by Cheon *et al.* [9]. Additionally, the gadget decomposition technique—introduced for the FV scheme by Bajard *et al.* [5] and subsequently adopted by others—reduces the noise introduced during key-switching by decomposing ring elements into shorter d-dimensional vectors with smaller entries. Han-Ki [21] applied this idea to the CKKS scheme [10], and combined it with the temporary modulus technique for further noise control. Kim *et al.* [23] expressed this CKKS key-switching procedure as an external product of the evaluation key (a $2 \times d$ matrix) and the gadget decomposition of the input ciphertext (a d-dimensional vector). Their formulation reduces key-switching to four steps: a gadget decomposition, its extension to the temporary modulus $Q \cdot P$, an inner product with the evaluation key, and a modulus lowering to Q.

In this work, we generalize this state-of-the-art decomposition key-switching technique [23] to an arbitrary number of input polynomials and secret keys. We present our Batched Key-Switching algorithm, which finds an affine ciphertext (with two components) under a target secret key that approximately opens to the linear combination of the n input polynomials with the n source secret keys. It captures many key-switching algorithms for CKKS presented in the literature. The term "batched" refers to the fact that key-switching for the n polynomials is performed at once via a single external product by an evaluation key of shape $2 \times nd$. The main advantage of batching the n key-switches is that the modulus reduction is performed only for the two components of the output ciphertext.

Our algorithm includes additional evaluation keys involving a secret key sk and transformations thereof, which we show to not leak any information under the RLWE hardness assumption alone. We also prove its correctness, provide bounds for its error growth, and show how this algorithm may be used for accumulating ciphertext multiplications and automorphisms. The latter kind of operations are a key part of the double-hoisted baby-step giant-step algorithm [7]

for homomorphically evaluating a linear transform, which we recast in terms of our batched algorithms in order to study its noise behavior. The implication of our results extends to two important parts of a CKKS bootstrapping procedure: its coefficients-to-slots and slots-to-coefficients steps.

The main drawback of our approach is that it requires handling a larger number of polynomials simultaneously, which leads to increased memory usage. In the case of batched multiplication, we postulate that our proposal is practical when the number of ciphertext factors is ≤ 4, even in applications such as DNN inference where the ring dimension $N \geq 2^{15}$. We give some practical examples of plynomial evaluations over ciphertexts to exemplify the use of our algorithm in this context when approximating nonlinear layers in DNN with polynomials.

In summary, our work includes the following contributions:

- We present the *Generalized Batched Decomposed Key-Switching* method (Algorithm 2), which extends the gadget decomposition key-switching technique [23] to multiple polynomial inputs. This algorithm can be instantiated to represent the most commonly used RNS-based key-switching algorithms appearing in the literature. We formally prove its security and correctness, and measure the error growth it incurs.
- We present a *General Batched Relinearization* method (Algorithm 3) that adds the decomposition technique to reduce noise and generalizes the multi-input ciphertext multiplication algorithm of Akherati *et al.* [2] by accepting ciphertexts of any degree in the secret-key indeterminate. We prove its correctness and error bounds following the blueprint of Han-Ki [21]. We demonstrate the algorithm on three polynomial approximation problems arising in private DNN inference applications.
- We give a concise yet complete description (Algorithm 5) of Bossuat *et al.*'s double-hoisted BSGS algorithm [7], discover one optimization concerning the automorphism applications during its giant steps, and provide to the best of our knowledge its first explicit error analysis in the literature.

The rest of this paper is organized as follows: Section 2 reviews the relevant background concepts related to our work. Section 3 introduces the Batched Key-Switching algorithm. Section 4 presents our General Batched Generalization method. Section 5 shows the use of our key-switching techniques within the double-hoisted BSGS matrix-vector multiplication algorithm. Section 6 presents our conclusions and discusses potential future work.

2 Background

In this section we review the CKKS encryption scheme [10] and its full-RNS variant [9]. We also study the gadget decomposition technique.

Notation. Bold lowercase and capital letters refer to vectors and matrices, respectively. If N is a power of two and $Q \geq 2$, let $\mathcal{R}$ and $\mathcal{R}_Q$ denote respectively the cyclotomic rings $\mathbb{Z}[X]/(X^N+1)$ and $\mathbb{Z}_Q[X]/(X^N+1)$. We identify an integer

in $\mathbb{Z}_Q$ by its representative in $(-Q/2, Q/2]$. For $1 \leq p \leq \infty$ and a polynomial $a \in \mathcal{R}$, we use $\|a\|_p$ to denote the ℓ_p norm of the coefficient vector of a. The notation $v \leftarrow \chi$ means to sample a value v from the probability distribution χ. The operation $\lfloor \cdot \rceil$ rounds its input to the nearest integer; it applies coefficient-wise to a real polynomial, and in case of ties, it rounds up. Let PPT be the class of decision problems solved by a probabilistic polynomial-time machine with error probability $< 1/2$, and let $\lambda \in \mathbb{N}$ be the security parameter.

The RLWE Problem. We recall the decision variant of the ring learning with errors problem [31] over the ring $\mathcal{R}_Q$, and its hardness assumption with respect to the distributions $\chi_{\text{key}}, \chi_{\text{err}}$ over $\mathcal{R}$: Sample $\mathsf{sk} \leftarrow \chi_{\text{key}}$. It is computationally hard to distinguish between two sets of $\lambda^{O(1)}$ samples (a_i, b_i) and $(a_i, (-a_i \cdot \mathsf{sk} + e_i) \bmod Q)$, where a_i, b_i are uniformly random in $\mathcal{R}_Q$, and $e_i \leftarrow \chi_{\text{err}}$.

The CKKS Scheme. The Cheon-Kim-Kim-Song scheme [10] is an approximate FHE scheme built on the hardness of the RLWE problem. It operates on vectors of complex numbers by encoding these into a polynomial form suitable for encrypted computation. Let ω be a primitive $(2N)$th root of unity, and define $\tau(f) = (f(\omega), f(\omega^3), \ldots, f(\omega^{N-1}))$ to be the canonical embedding from $\mathbb{R}[X]/(X^N + 1)$ into $\mathbb{C}^{N/2}$. To obtain the plaintext representation of a message $\boldsymbol{m} \in \mathbb{C}^{N/2}$, apply the inverse of τ to $\boldsymbol{m}$, multiply the resulting polynomial by a given scaling factor Δ, and round its coefficients to the nearest integer. The result is a polynomial in $\mathcal{R}$ that—following a procedure that we shall explain shortly—may be encrypted into a ciphertext, namely a polynomial in the ring $\mathcal{R}_Q[S] = \mathbb{Z}_Q[X, S]/(X^N + 1)$.[1] The CKKS scheme provides the following elementary operations on keys and ciphertexts:

$\mathsf{Setup}(N, Q, P, \lambda, \eta)$. Take a power of two N, coprime integers Q and P, a security level λ, and a bit precision parameter η as input. Define $\tilde{Q} := QP$. Select a secret key distribution χ_{key}, an encryption key distribution χ_{enc}, and an error distribution χ_{err} over $\mathcal{R}$ that guarantee λ-bit security.

KeyGen. Sample the secret key $\mathsf{sk} \leftarrow \chi_{\text{key}}$, and construct the public key $\mathsf{pk} = a(S - \mathsf{sk}) + e \in \mathcal{R}_Q[S]$, where a is uniformly random in $\mathcal{R}_Q$ and $e \leftarrow \chi_{\text{err}}$.

$\mathsf{Encrypt}(\mathsf{pt}, \mathsf{pk})$. Given a plaintext $\mathsf{pt} \in \mathcal{R}$, sample $v \leftarrow \chi_{\text{enc}}$ and $e_0, e_1 \leftarrow \chi_{\text{err}}$; then output the ciphertext $\mathsf{ct}(S) = \mathsf{pk} + \mathsf{pt} + e_1 S + e_0 \in \mathcal{R}_Q[S]$.

$\mathsf{Decrypt}(\mathsf{ct}, \mathsf{sk})$. Evaluate a given ciphertext $\mathsf{ct}(S) \in \mathcal{R}_Q[S]$ at $S = \mathsf{sk}$, mod Q.

The full-RNS variant of the CKKS scheme [9] uses a residue number system (RNS) to represent an element of $\mathbb{Z}_Q$ as the list of its residues modulo some coprime integers $q_0, q_1, \ldots, q_L$ whose product is Q. The isomorphism $\mathcal{R}_Q \cong \mathcal{R}_{q_0} \times \cdots \times \mathcal{R}_{q_L}$ extends this idea in a coefficient-wise manner, allowing us to split a polynomial in $\mathcal{R}_Q$ into $L + 1$ polynomials that feature smaller coefficients (and thus are easier to compute with). For reasons that we discuss later, some CKKS operations work over the "extended" ring $\mathcal{R}_{\tilde{Q}}$ instead of over $\mathcal{R}_Q$, and so it becomes necessary to convert the RNS representation of an integer between two coprime bases. Let

[1] Such a ciphertext can also be regarded as a tuple of polynomials in $\mathcal{R}_Q$, but the multivariate polynomial representation is more convenient for our purposes.

$(a_0, \ldots, a_{l-1})$ be the RNS representation of an integer $a \in \mathbb{Z}_Q$ with respect to the basis $(q_0, \ldots, q_{l-1})$, for some $0 \le l \le L+1$. The following formula approximately transforms this RNS representation into one over another basis $(p_1, \ldots, p_k)$ of coprime moduli:

$$(\tilde{a}_1, \ldots, \tilde{a}_k) = \left(\sum_{j=0}^{l-1} (a_j \cdot \hat{q}_j^{-1} \bmod q_j) \cdot \hat{q}_j \pmod{p_i} \right)_{i=1}^{k}, \quad \text{where } \hat{q}_j = \frac{Q}{q_j}. \quad (1)$$

More precisely, $(\tilde{a}_1, \ldots, \tilde{a}_k)$ is the RNS representation of an integer $\tilde{a} = a + Q \cdot e$ such that $\tilde{a} \equiv a \pmod{Q}$ and $|e| \le l/2$. In this way, the RNS extension of a from $\mathbb{Z}_Q$ to $\mathbb{Z}_{\tilde{Q}}$ is the concatenation of the RNS representations $(a_0, \ldots, a_{l-1})$ and $(\tilde{a}_1, \ldots, \tilde{a}_k)$. Proceeding in the reverse direction, to approximately lower the RNS representation of $\tilde{a} \in \mathbb{Z}_{\tilde{Q}}$ back to the basis $(q_0, \ldots, q_{l-1})$, use the basis conversion formula (1) to convert $(\tilde{a}_1, \ldots, \tilde{a}_k)$ from the basis $(p_1, \ldots, p_k)$ into $(c_0, \ldots, c_{l-1})$ in the basis $(q_0, \ldots, q_{l-1})$; then compute $(b_0, \ldots, b_{l-1}) = (\lfloor P^{-1} \cdot (a_j - c_j) \rceil)_{0 \le j < l}$, which represents an integer $b \in \mathbb{Z}_Q$ such that $|b - P^{-1} \cdot \tilde{a}| \le k/2$.

The full-RNS variant of CKKS manages the magnitude of ciphertexts through a *rescaling* procedure. A ciphertext $\mathsf{ct}(S)$ starts as a polynomial in the ring $\mathcal{R}_Q[S]$ and has an associated scaling factor Δ. Homomorphic operations progressively grow the effective scale of $\mathsf{ct}(S)$ and consume the coefficient modulus $Q = q_0 \cdots q_L$. Rescaling reverts these effects by lowering the RNS representation of $\mathsf{ct}(S)$ from the basis $(q_0, \ldots, q_l)$ down to $(q_0, \ldots, q_{l-1})$ for $0 \le \ell < L$, i.e., by dropping the last modulus q_l. Defining $Q_l = \prod_{k=0}^{l} q_k$, this operation is formalized as follows:

$\mathsf{Rescale}(\mathsf{ct}, \Delta)$. Given a ciphertext $\mathsf{ct}(S) \in \mathcal{R}_{Q_l}$ at level $0 < l \le L$ and scale Δ, output the ciphertext $\mathsf{ct}'(S) = \lfloor q_l^{-1} \cdot \mathsf{ct}(S) \rceil \in \mathcal{R}_{Q_{l-1}}$ at level $l-1$ and scale Δ/q_l.

The discussion above shows that $\mathsf{ct}'(\mathsf{sk}) = \lfloor q_l^{-1} \cdot \mathsf{ct}(\mathsf{sk}) \rceil + e(\mathsf{sk})$ for some $e(S) \in \mathbb{Q}[X, S]/(X^N + 1)$ satisfying $\|e(\mathsf{sk})\|_\infty \le B_{\mathrm{rs}} := (1 + \|\mathsf{sk}\|_1)/2$. Rescaling is an essential part of a multiplication circuit, whose discussion we defer to Section 4.

Gadget Decompositions. The gadget decomposition technique is a useful tool for cutting down noise growth during homomorphic computations. Genise *et al.* [16] define a $\mathbb{Z}_Q$*-gadget of quality* β as a vector $\boldsymbol{g} \in \mathbb{Z}_Q^d$ for which there exists a function $h \colon \mathbb{Z}_Q \to \mathbb{Z}_Q^d$ such that $\boldsymbol{g}^T h(a) = a$ and $\|h(a)\|_\infty \le \beta$ for every integer $a \in \mathbb{Z}_Q$. Kim *et al.* [23] described a practical method for constructing gadget vectors from an RNS basis $\mathcal{B} = \{q_0, \ldots, q_L\}$: Partition $\mathcal{B}$ into nonempty subsets $\mathcal{B}_1, \ldots, \mathcal{B}_d$, and define the *digits* $D_j = \prod_{q \in \mathcal{B}_j} q$ for $1 \le j \le d$. By Equation (1), the vector $\boldsymbol{g} \in \mathbb{Z}_Q^d$ with entries $g_j = ((Q/D_j)^{-1} \bmod D_j) \cdot (Q/D_j)$ is a $\mathbb{Z}_Q$-gadget of quality $\beta = \frac{1}{2} \max_{1 \le j \le d} D_j$. The relevant decomposition function h maps each element of $\mathbb{Z}_Q$ into its RNS representation in the basis $(D_1, \ldots, D_d)$.

In this setting, the *external product* $\boldsymbol{u} \boxdot v$ of a vector $\boldsymbol{u} \in \mathbb{Z}_{\tilde{Q}}$ by an integer $v \in \mathbb{Z}_Q$ is defined as $\lfloor P^{-1} \cdot (\boldsymbol{u}^T h(v) \bmod \tilde{Q}) \rceil \pmod{Q}$. This operation arises naturally in the key-switching procedure of CKKS, the main subject of this paper. It generalizes to matrices $\boldsymbol{U} \in \mathbb{Z}_{\tilde{Q}}^{k \times d}$ as $\boldsymbol{U} \boxdot v = \left[\boldsymbol{u}_1 \boxdot v, \ldots, \boldsymbol{u}_k \boxdot v \right]^T$, and to polynomials $v \in \mathcal{R}_Q$ by decomposing each coefficient independently.

The NTT. Suppose that $q_i \equiv 1 \pmod{2N}$ for every $0 \le i \le L$, and let ω_i be a primitive $(2N)$th root of unity modulo q_i. A Number Theoretic Transform maps a polynomial $a \in \mathcal{R}_{q_i}$ into the evaluation vector $\hat{\boldsymbol{a}} = (a(\omega_i), a(\omega_i^3), \ldots, a(\omega_i^{2N-1})) \in \mathbb{Z}_{q_i}^N$. Since the NTT is a discrete Fourier transform, it reduces polynomial multiplication to entry-wise multiplication of evaluation vectors. A polynomial is said to be in *coefficient form* when represented by its coefficients, and in *evaluation form* when represented by its NTT vector. Applying an NTT to a polynomial in $\mathcal{R}_{Q_l}$ amounts to applying the NTT to each component of its RNS representation in the basis $(q_0, \ldots, q_l)$. It will be convenient to define the norm $|||a|||_\infty = \|\hat{\boldsymbol{a}}\|_\infty$, and to remember the inequalities

$$|||a \cdot b|||_\infty \le |||a|||_\infty |||b|||_\infty, \qquad |||a|||_\infty \le \|a\|_1, \qquad \|a\|_\infty \le |||a|||_\infty,$$

for $a, b \in \mathcal{R}_Q$; the last one holds true only because N is a power of 2 (see [12]).

3 Batched Key-Switching

In this section, we evolve from the state–of–the–art decomposition key-switching technique into a generalization that operates upon an arbitrary number of input polynomials and secret keys, which we call *Batched* Key-Switching. Our proposed algorithm performs the switching for *all* the polynomials by just *one* external product by a matrix in $\mathcal{R}_{\tilde{Q}}^{2\times nd}$, hence performing a single modulus lowering.

A number of operations in the CKKS scheme proceed in two steps: First, they transform one or more ciphertexts into an affine-linear form $f(S, S') = b + aS + rS'$, where b, a, $r \in \mathcal{R}_Q$. Second, they apply a key-switch operation to the coefficient r to obtain a ciphertext over the single secret-key indeterminate S. For example, to obtain the product of two affine ciphertexts $\mathsf{ct}(S)$ and $\mathsf{ct}'(S)$ that encrypt messages under a common secret key sk, we compute the form $f(S, S^2) = \mathsf{ct}(S) \cdot \mathsf{ct}'(S)$ and subsequently replace its term rS^2 by $\boldsymbol{U} \boxdot r$, where $\boldsymbol{U}$ is an evaluation key from sk^2 to sk. Many CKKS circuits conform to a more general version of this pattern, in which the first step combines the input ciphertexts into a larger affine-linear form $f(S, S'_1, \ldots, S'_n) = b + aS + \sum_{i=1}^n r_i S'_i$ whose coefficients are all in $\mathcal{R}_Q$; the second step then takes the form

$$\begin{array}{l} \text{Initialize } \mathsf{ct}(S) \text{ to some affine ciphertext in } \mathcal{R}_Q[S]. \\ \text{For } 1 \le i \le n, \text{ add } \boldsymbol{U}_i \boxdot r_i \text{ to } \mathsf{ct}(S). \end{array} \tag{2}$$

Here $\boldsymbol{U}_1, \ldots, \boldsymbol{U}_n$ are evaluation keys constructed from the same gadget vector $\boldsymbol{g} \in \mathbb{Z}_Q^d$. The loop in Program (2) requires a significant amount of computation, because each external product evaluation involves a gadget decomposition, several NTT operations, and $d + 2$ RNS reconstructions. Furthermore, since the last two of these reconstructions need an integer rounding operation to reduce the modulus space, they introduce a nonnegligible amount of noise into $\mathsf{ct}(S)$. These deficiencies have led many authors dealing with particular instances of Program (2) to suggest taking the modulus lowering step out of the loop. In this way, the

circuit becomes

$$\begin{array}{l}\text{Initialize } \tilde{\mathsf{ct}}(S) \text{ to some affine ciphertext in } \mathcal{R}_{\tilde{Q}}[S].\\ \text{For } 1 \le i \le n,\\ \quad \text{Construct the gadget decomposition } \boldsymbol{d}_i \text{ of } r_i \text{ with respect to } \boldsymbol{g}.\\ \quad \text{Lift } \boldsymbol{d}_i \text{ to its RNS representation } \tilde{\boldsymbol{d}}_i \text{ over } \mathcal{R}_{\tilde{Q}}^d.\\ \quad \text{Add the affine polynomial } \begin{bmatrix}1 & S\end{bmatrix} \boldsymbol{U}_i \tilde{\boldsymbol{d}}_i \text{ to } \tilde{\mathsf{ct}}(S).\\ \text{Reduce } \tilde{\mathsf{ct}}(S) \text{ back to } \mathcal{R}_Q[S] \text{ to obtain } \mathsf{ct}(S).\end{array} \tag{3}$$

Notice that the addition of the key-switched r_i's now proceeds over the extended ring $\mathcal{R}_{\tilde{Q}}[S]$, but this is more than ample compensation for the savings of $2n-2$ RNS reconstructions and a reduction of noise due to rounding from $O(n)$ to $O(1)$ in the ℓ_∞-norm sense.

There are circumstances where we face a slightly different problem: given a ciphertext $\mathsf{ct}(S) = b + aS$ in $\mathcal{R}_Q[S]$, the objective now is to obtain multiple affine ciphertexts $\mathsf{ct}_i(S)$ that decrypt under different secret keys:

$$\begin{array}{l}\text{Initialize } \mathsf{ct}'(S) \text{ to some affine ciphertext}\\ \quad \text{in } \mathcal{R}_Q[S], \text{ possibly depending on } \mathsf{ct}(S).\\ \text{For } 1 \le i \le n, \text{ set } \mathsf{ct}_i(S) \text{ to } \mathsf{ct}'(S) + \boldsymbol{U}_i \boxdot a.\end{array} \tag{4}$$

Notice that all external products take the polynomial a as input, and thus perform the same d RNS reconstructions in order to construct its gadget decomposition. We may avoid the redundant computation by extracting this step out of the loop:

$$\begin{array}{l}\text{Initialize } \mathsf{ct}'(S) \text{ to some affine ciphertext in } \mathcal{R}_Q[S].\\ \text{Construct the gadget decomposition } \boldsymbol{d} \text{ of } a \text{ in the basis } \boldsymbol{g}.\\ \text{Lift } \boldsymbol{d} \text{ to its RNS representation } \tilde{\boldsymbol{d}} \text{ over } \mathcal{R}_{\tilde{Q}}.\\ \text{For } 1 \le i \le n,\\ \quad \text{Set } \tilde{\mathsf{ct}}_i(S) \text{ to the affine polynomial } \begin{bmatrix}1 & S\end{bmatrix} \boldsymbol{U}_i \tilde{\boldsymbol{d}}.\\ \quad \text{Reduce } \tilde{\mathsf{ct}}_i(S) \text{ back to } \mathcal{R}_Q[S] \text{ to obtain } \mathsf{ct}_i(S).\\ \quad \text{Add } \mathsf{ct}'(S) \text{ to } \mathsf{ct}_i(S).\end{array} \tag{5}$$

The change from Program (4) to (5) poses absolutely no effect on noise growth, but it sometimes gives us the opportunity to reduce the amount of noise introduced by subsequent operations, namely by applying these to the intermediate $\tilde{\mathsf{ct}}_i(S)$ over the extended ring $\mathcal{R}_{\tilde{Q}}[S]$, instead of to $\mathsf{ct}_i(S)$. (Of course, this usually leads to a slight increase in execution time and/or space.)

The two "hoisting" transformations we have just seen turn out to apply in a wide variety of situations, which makes them good candidates for integration as optimization passes into homomorphic encryption compilers like HEaaN.MLIR [29] and Google HEIR [4]. To illustrate this point, Sections 4 and 5 discuss two particular, yet popular, applications from the literature. Their running times have already been analyzed in detail (and the complexity analyses extend readily to the general case, as we shall see) but, to the best of our knowledge, other important aspects such as their noise growth behavior remain unknown. Our present goal is to settle some of these matters by studying Programs (3) and (5) in full generality; later, we will discuss their implications on the applications.

Let us begin by describing a procedure that generates the necessary evaluation keys in batch, Algorithm 1.

Algorithm 1: Evaluation Key Generation

Given secret keys $\mathsf{sk}'_1, \mathsf{sk}'_2, \ldots, \mathsf{sk}'_n, \mathsf{sk}$ in $\mathcal{R}$, and a gadget vector $\boldsymbol{g} \in \mathbb{Z}_Q^d$, this algorithm generates an evaluation key $\boldsymbol{U} \in \mathcal{R}_{\tilde{Q}}^{2\times nd}$ necessary for the key-switching procedure in Algorithm 2.

Express the matrix $\boldsymbol{U}$ in the block form $\left[\begin{smallmatrix} \boldsymbol{u}_{0,1} & \cdots & \boldsymbol{u}_{0,n} \\ \boldsymbol{u}_{1,1} & \cdots & \boldsymbol{u}_{1,n} \end{smallmatrix}\right]$, where each $\boldsymbol{u}_{i,j}$ is a row vector in $\mathcal{R}_{\tilde{Q}}^d$. Select the n coefficients of each entry of $\boldsymbol{u}_{1,i}$ uniformly at random, for $1 \le i \le n$. Then generate a row vector $\boldsymbol{e}_i \in \mathcal{R}^d$ whose entries have coefficients sampled from χ_{err}, and set $\boldsymbol{u}_{0,i} := -\mathsf{sk} \cdot \boldsymbol{u}_{1,i} + P \cdot \mathsf{sk}'_i \cdot \boldsymbol{g}^T + \boldsymbol{e}_i$ modulo $\tilde{Q}$.

Bundling several evaluation keys into a single matrix does not compromise security, as we prove in Theorem 1.

Theorem 1. *Under the RLWE hardness assumption, an evaluation key $\boldsymbol{U}$ generated by an honest party via Algorithm 1 is computationally indistinguishable from a $2 \times nd$ matrix of independently and uniformly distributed polynomials in $\mathcal{R}_{\tilde{Q}}$, provided that n and d are polynomial in λ.*

Proof. The goal is to show that the nd columns of $\boldsymbol{U}$ have a joint distribution $\mathcal{D}_0$ that is computationally indistinguishable from that of nd independent and uniformly random pairs of polynomials in $\mathcal{R}_{\tilde{Q}}$. For $1 \le k \le nd$, define $\mathcal{D}_k$ to be the same distribution as $\mathcal{D}_{k-1}$, but with the kth pair being uniformly random and independent of the others; in terms of this notation, we need to prove that $\mathcal{D}_0$ and $\mathcal{D}_{nd}$ are indistinguishable.

Let $\mathcal{A}$ be a PPT adversary that can distinguish between two consecutive distributions $\mathcal{D}_k$ and $\mathcal{D}_{k-1}$ with advantage ε, for any $1 \le k \le nd$. We shall now construct a PPT reduction $\mathcal{B}$ that uses $\mathcal{A}$ to break the RLWE assumption over $\mathcal{R}_{\tilde{Q}}$ with the same advantage ε. The reduction $\mathcal{B}$ receives a challenge pair (a, b) that is either a RLWE sample (which satisfies $b = -a \cdot \mathsf{sk} + e$ for some $e \leftarrow \chi_{\mathrm{err}}$), or a pair of polynomials sampled uniformly from $\mathcal{R}_{\tilde{Q}}$. Next, it samples secret keys $\mathsf{sk}'_{\lfloor (k-1)/d \rfloor + 1}, \ldots, \mathsf{sk}'_n, \mathsf{sk} \leftarrow \chi_{\mathrm{key}}$, and partly simulates the evaluation key generation process in Algorithm 1. More precisely, it constructs a matrix $\boldsymbol{V} \in \mathcal{R}_{\tilde{Q}}^{2\times nd}$ such that

- The first $k-1$ columns consist of uniformly random polynomials in $\mathcal{R}_{\tilde{Q}}$.
- Columns $j = k+1, \ldots, nd$ have the form $\left[v_{0,j}\ v_{1,j}\right]^T$, where $v_{1,j}$ is uniformly random in $\mathcal{R}_{\tilde{Q}}$, and $v_{0,j} := -\mathsf{sk} \cdot v_{1,j} + P \cdot \mathsf{sk}'_{\lfloor (j-1)/d \rfloor + 1} \cdot \boldsymbol{g}_{(j-1) \bmod d+1} + e_j$ with $e_j \leftarrow \chi_{\mathrm{err}}$. These are valid evaluation keys.
- The kth column $\left[v_{0,k}\ v_{1,k}\right]^T$ satisfies $v_{1,k} := a$ and $v_{0,k} := b + P \cdot \mathsf{sk}'_{\lfloor (k-1)/d \rfloor + 1} \cdot \boldsymbol{g}_{(k-1) \bmod d+1}$.

Note that $v_{0,k} + \mathsf{sk} \cdot v_{1,k} = b + \mathsf{sk} \cdot a + P \cdot \mathsf{sk}'_{\lfloor (k-1)/d \rfloor + 1} \cdot \boldsymbol{g}_{(k-1) \bmod d+1}$ is a valid evaluation key if and only if (a, b) is an RLWE sample. (When b is uniformly random, it acts as a blinding mask for the other term in $v_{0,k}$, which makes the kth column of $\boldsymbol{V}$ uniformly random.) The reduction $\mathcal{B}$ finishes by outputting the same bit as $\mathcal{A}$ on input $\boldsymbol{V}$. The previous construction guarantees that $\boldsymbol{V}$ has the distribution $\mathcal{D}_k$ if (a, b) is a RLWE sample, and $\mathcal{D}_{k-1}$ otherwise. Consequently, $\mathcal{B}$ breaks the RLWE problem with the same advantage that $\mathcal{A}$ has for distinguishing between two consecutive distributions. So ε is negligible by the RLWE hardness assumption.

Finally, an adversary that can distinguish between $\mathcal{D}_0$ and $\mathcal{D}_{nd}$ has advantage

$$\begin{aligned} &\left| \Pr_{h \leftarrow \mathcal{D}_{nd}} (\mathcal{A}(h) = 1) - \Pr_{h \leftarrow \mathcal{D}_0} (\mathcal{A}(h) = 1) \right| \\ &\qquad \leq \sum_{k=1}^{nd} \left| \Pr_{h \leftarrow \mathcal{D}_k} (\mathcal{A}(h) = 1) - \Pr_{h \leftarrow \mathcal{D}_{k-1}} (\mathcal{A}(h) = 1) \right| \leq nd\varepsilon, \end{aligned}$$

which is also negligible, since nd is polynomial in λ. □

Using the evaluation key generated by Algorithm 1, the method described in Algorithm 2 applies the key-switch operation to a batch of polynomials (mimicking Program (3)).

Algorithm 2: Batched Key-Switching

Let $\mathsf{sk}'_1, \mathsf{sk}'_2 \ldots, \mathsf{sk}'_n, \mathsf{sk}$ be secret keys in $\mathcal{R}$. Given polynomials $r_1, \ldots, r_n \in \mathcal{R}_Q$ and $\tilde{b} \in \mathcal{R}_{\tilde{Q}}$ in coefficient form, and an evaluation key $\boldsymbol{U} \in \mathcal{R}_{\tilde{Q}}^{2 \times nd}$ from $(\mathsf{sk}'_1, \ldots, \mathsf{sk}'_n)$ to sk produced by Algorithm 1 for a gadget vector $\boldsymbol{g} \in \mathbb{Z}_{\tilde{Q}}^d$, this algorithm finds an affine ciphertext $\mathsf{ct}(S) \in \mathcal{R}_Q[S]$ in coefficient form such that $\mathsf{ct}(\mathsf{sk}) \approx r_1 \mathsf{sk}'_1 + \cdots + r_n \mathsf{sk}'_n + P^{-1} \cdot \tilde{b} \pmod{Q}$.

1. [Decompose.] For $1 \leq i \leq n$, construct the gadget decomposition $\boldsymbol{d}_i \in \mathcal{R}_Q^d$ of r_i with respect to the basis vector $\boldsymbol{g}$.
2. [Extend.] For $1 \leq i \leq n$, find the RNS representation $\tilde{\boldsymbol{d}}_i$ of $\boldsymbol{d}_i$ over $\mathcal{R}_{\tilde{Q}}^d$.
3. [NTT.] Convert each $\tilde{\boldsymbol{d}}_i$ into its evaluation form, for $1 \leq i \leq n$.
4. [Batch.] Compute the bilinear form

$$\tilde{\mathsf{ct}}(S) := \begin{bmatrix} 1 & S \end{bmatrix} \boldsymbol{U} \begin{bmatrix} \tilde{\boldsymbol{d}}_1^T & \cdots & \tilde{\boldsymbol{d}}_n^T \end{bmatrix}^T = \sum_{i=1}^{n} \boldsymbol{u}_{0,i} \tilde{\boldsymbol{d}}_i + S \sum_{i=1}^{n} \boldsymbol{u}_{1,i} \tilde{\boldsymbol{d}}_i \mod \tilde{Q}.$$

5. [INTT.] Convert $\tilde{\mathsf{ct}}(S)$ into its coefficient form.
6. [Add bias.] Add the polynomial $\tilde{b}$ to $\tilde{\mathsf{ct}}(S)$.
7. [Lower.] Output the ciphertext $\mathsf{ct}(S) = \lfloor P^{-1} \cdot \tilde{\mathsf{ct}}(S) \rceil$.

Note that Algorithm 2 is written in terms of the full modulus Q, but it is a straightforward matter to adapt it to leveled inputs—with careful attention

in step 1 to construct the gadget decompositions correctly, however. The full complexity analysis of Algorithm 2 can be carried out in an analogous way as Kim *et al.* [23]; we only mention that in some cases $\tilde{b}$ is given in evaluation form, and then it is advantageous to swap the order of steps 5 and 6 to save on one INTT operation.

Theorem 2 shows the correctness of Algorithm 2, and provides a bound on the amount of noise that it introduces.

Theorem 2. *Let $Q = q_0 \cdots q_l$ and $P = p_1 \cdots p_k$ be coprime products of coprime integers, let $\boldsymbol{g} \in \mathbb{Z}_Q^d$ be a gadget vector, and let $Q = D_1 \cdots D_d$ be the digit decomposition used to construct $\boldsymbol{g}$. Define $C := Nn\varepsilon_{\mathrm{f}}(l+1)\max_{1\le j\le d} D_j$, where ε_{f} is an upper bound on the ℓ_∞ norm of the error term in a fresh encryption. If $P > C$, then Algorithm 2 produces a ciphertext $\mathsf{ct}(S) \in \mathcal{R}_Q[S]$ that opens to $\mathsf{ct}(\mathsf{sk}) \equiv \sum_{i=1}^n r_i \cdot \mathsf{sk}_i' + P^{-1} \cdot \tilde{b} + \varepsilon(\mathsf{sk}) \bmod Q$, where $\|\varepsilon(\mathsf{sk})\|_\infty \le \frac{1}{2}(1+k+\|\mathsf{sk}\|_1 k)$.*

Proof. After step 4, the ciphertext $\tilde{\mathsf{ct}}$ opens to $\tilde{\mathsf{ct}}(\mathsf{sk}) \equiv \sum_{i=1}^n (\boldsymbol{u}_{0,i} + \mathsf{sk} \cdot \boldsymbol{u}_{1,i})\, \tilde{\boldsymbol{d}}_i \pmod Q$, where $\boldsymbol{u}_{0,i} + \boldsymbol{u}_{1,i} S$ is an encryption of sk_i' under sk with respect to the gadget basis $\boldsymbol{g}$. Hence:

$$\begin{aligned}\tilde{\mathsf{ct}}(\mathsf{sk}) &= \sum_{i=1}^n \left(-\mathsf{sk} \cdot \boldsymbol{u}_{1,i} + P \cdot \mathsf{sk}_i' \cdot \boldsymbol{g}^T + \boldsymbol{e}_i + \mathsf{sk} \cdot \boldsymbol{u}_{1,i}\right) \tilde{\boldsymbol{d}}_i \\ &= \sum_{i=1}^n \left(P \cdot \mathsf{sk}_i' \cdot \boldsymbol{g}^T + \boldsymbol{e}_i\right) \tilde{\boldsymbol{d}}_i \mod \tilde{Q}.\end{aligned}$$

All the polynomial products in step 4 are performed in the evaluation domain; by Minkowski's inequality and the properties of the evaluation norm $|||\cdot|||$, the error term $\tilde{e} := \sum_{i=1}^n \boldsymbol{e}_i \tilde{\boldsymbol{d}}_i$ of $\tilde{\mathsf{ct}}(S)$ satisfies the bound:

$$|||\tilde{e}|||_\infty \le \sum_{i=1}^n |||\boldsymbol{e}_i \tilde{\boldsymbol{d}}_i|||_\infty = \sum_{i=1}^n \sum_{j=1}^d |||(\boldsymbol{e}_i)_j|||_\infty |||(\tilde{\boldsymbol{d}}_i)_j|||_\infty. \tag{6}$$

Recall that vector $\tilde{\boldsymbol{d}}_i$ is the extension to $\mathcal{R}_{\tilde{Q}}^d$ of the gadget decomposition $\boldsymbol{d}_i$ in the basis $\boldsymbol{g} \in \mathbb{Z}_Q^d$. Denote by L_j the length of digit D_j, namely the number of moduli $q_0, \ldots, q_l$ that comprise it. We know in particular that $L_1 + \cdots + L_d = l+1$, because the digits $D_1, \ldots, D_d$ correspond to a partition of the q_i. After step 2—which uses the fast basis conversion technique to extend the RNS representation of each polynomial $(\boldsymbol{d}_i)_j \in \mathcal{R}_{D_j}$ to $(\tilde{\boldsymbol{d}}_i)_j \in \mathcal{R}_{\tilde{Q}}$—the inequality $\|(\tilde{\boldsymbol{d}}_i)_j\|_1 \le N L_j D_j/2$ holds. Applying this bound together with $\|\boldsymbol{e}_i\|_\infty \le \varepsilon_{\mathrm{f}}$ and $|||(\boldsymbol{d}_i)_j|||_\infty \le \|(\tilde{\boldsymbol{d}}_i)_j\|_1$ to Eq. (6) yields

$$|||\tilde{e}|||_\infty \le \tfrac{1}{2} Nn\varepsilon_{\mathrm{f}} \sum_{j=1}^d L_j D_j \le \tfrac{1}{2} Nn\varepsilon_{\mathrm{f}}(l+1) \max_{1\le j\le d} D_j = \frac{C}{2}.$$

Step 5 brings $\tilde{\mathsf{ct}}(S)$ into coefficient form. The previous bound still holds for $\|\tilde{e}\|_\infty$, because N is a power of two. Finally, step 7 outputs a ciphertext $\mathsf{ct}(S) \in \mathcal{R}_Q[S]$

such that

$$\mathsf{ct}(\mathsf{sk}) \equiv \sum_{i=1}^{n} \mathsf{sk}'_i \cdot \boldsymbol{g}^T \tilde{\boldsymbol{d}}_i + P^{-1} \cdot \tilde{b} + P^{-1} \cdot \tilde{e} + \varepsilon_{\mathrm{r}}(\mathsf{sk}) \pmod{Q}, \tag{7}$$

where the polynomial $\varepsilon_{\mathrm{r}}(S) := \varepsilon_0 + \varepsilon_1 S \in \mathbb{Q}[S, X]/(X^N + 1)$ contains the rounding residuals. The first two terms on the right-hand side of Eq. (7) are precisely the result of key-switching without error, because $\boldsymbol{d}_i$ is the gadget decomposition of r_i in the basis $\boldsymbol{g}$. It follows from $\|\tilde{e}\|_\infty \leq C/2$ and the assumption $P > C$ that $\|P^{-1} \cdot \tilde{e}\|_\infty < 1/2$. And since $\|\varepsilon_0\|_\infty$ and $\|\varepsilon_1\|_\infty$ are both at most $k/2$,[2] Minkowski's and Young's convolution inequalities yield

$$\|\varepsilon_{\mathrm{r}}(\mathsf{sk})\|_\infty \leq \|\varepsilon_0\|_\infty + \|\mathsf{sk}\|_1 \|\varepsilon_1\|_\infty \leq \tfrac{1}{2}(1 + \|\mathsf{sk}\|_1)k.$$

The triangle inequality then implies that the total approximation error is

$$\|\varepsilon(\mathsf{sk})\|_\infty \leq \|P^{-1} \cdot \tilde{e}\|_\infty + \|\varepsilon_{\mathrm{r}}(\mathsf{sk})\|_\infty \leq \tfrac{1}{2}(1 + k + \|\mathsf{sk}\|_1 k). \qquad \square$$

Note that in the common case that χ_{key} is the sparse ternary distribution $\mathcal{H}(h)$, in which a sampled polynomial features exactly h nonzero coefficients, each set to either $+1$ or -1, we have $\|\mathsf{sk}\|_1 = h$.

Exchanging sk with sk'_i in the description of Algorithm 1 leads to a procedure that generates an evaluation key with which Program (5) may key-switch a single polynomial from one secret key into several distinct keys. It is easy to see that Theorem 1 still holds after this modification, i.e., that the evaluation key remains computationally indistinguishable from random. A noise analysis of this batched $\mathsf{sk} \to (\mathsf{sk}'_1, \ldots, \mathsf{sk}'_n)$ key-switching operation may be performed in exactly the same manner as in the proof of Theorem 2, and, in fact, that result applies to each output ciphertext $\mathsf{ct}_i(S)$ when $n = 1$.

Curiously, if the sk'_i are the result of applying automorphisms to sk, then it is possible to switch among the two forms of the evaluation key $\boldsymbol{U}$ without knowledge of the secret key sk; see Section 5.

4 General Batched Relinearization

High-order polynomials are used to approximate nonlinear functions in the context of private DNN inference with FHE. This requires numerous ciphertext multiplications, which involve the relinearization of their outputs to obtain a ciphertext affine in the secret-key indeterminate S. Since this process is very costly, previous works have proposed to move the location of the relinearization steps. For example, Akherati *et al.* [2] described a key-switching algorithm for any number of input RNS-CKKS ciphertexts, by including additional evaluation keys, just as we do in this work. Our general batched relinearization proposes the same idea, but it also adds the decomposition technique and extends it to ciphertext inputs

[2] In the original proof of Han–Ki [21, Theorem 2], this bound is missing the factor k.

of arbitrary degrees in S. This allows us to perform several multiplications before relinearization while reducing the number of modular lowerings. Using our method, we only perform 2 basis conversions in this lowering step, independently of the number c of ciphertexts that are multiplied, instead of the $c-1$ conversions with the standard binary tree reduction. This not only positively impacts performance, but also the amount of noise accumulated throughout the computations, as we analyze in Theorem 3 in Section 4.2. Concretely, we analyze the error growth and compare it to the standard process of relinearizing after each multiplication. First, we compare both approaches for $c = 4$ ciphertexts, continuing in a natural way the case $c = 3$ already studied by Akherati *et al.* [1]. Next, we analyze how the error scales to an arbitrary number of ciphertexts $c \geq 2$. Our approach includes some additional amount of error at the end, while the standard procedure adds the equivalent at each individual multiplication.

The disadvantage of delaying relinearization is that both the degree of the ciphertexts in the secret-key indeterminate S and the amount of key-switching material grow considerably, but recent and emerging memory architectures such as CXL-attached memory are candidates to provide sufficient space to store temporary variables for typical private machine-learning inference use cases, as early works suggested, e.g., leveraging Intel Optane Persistent Memory [25]. Technologies such as near-memory processing or processing-in-memory could also alleviate the additional data transfers required by the increased memory footprint [6].

The delayed or "lazy" relinearization technique is already supported by some implementations of the CKKS scheme. For example, SEAL [32] supports the multiplication of two ciphertexts $\mathsf{ct}_1(S), \mathsf{ct}_2(S) \in \mathcal{R}_Q[S]$ of arbitrary degrees k_1, k_2 in sk. It computes each of the k_1+k_2+1 coefficients of $\mathsf{ct}_1\mathsf{ct}_2$ one by one, performing $k_1 \cdot k_2$ modular multiplications in total. OpenFHE [3] and Lattigo [15] work in the same way, and Lattigo already employs the RNS decomposition technique. Although these CKKS implementations support flexible multiplications, only a limited number of CKKS applications actually employ them in practice. For example, EVA and Orion [13, 14] are two popular FHE DNN inference engines that relinearize after every ciphertext multiplication. Moreover, all the implementations we have found are restricted to two input ciphertexts, instead of the arbitrary number of ciphertexts that we propose. Operations such as polynomial evaluation often require exponentiation through repeated multiplications, for example via a recursive approach. Thus, the cost of relinearizing at each step clearly affects the performance of these applications. Section 4.3 shows how to apply our algorithm for computing polynomial approximations of three nonlinear operations used in DNN inference: the reciprocal, exponential, and GELU functions.

4.1 Scheme Description

Let $\mathsf{ct}_1, \ldots, \mathsf{ct}_c$ be ciphertexts of respective degrees $k_1, \ldots, k_c$ in the secret-key variable S, all defined at the same level. Each ciphertext ct_i is a multivariate polynomial over the ring $\mathcal{R}_Q[S]$, hence we may express $\mathsf{ct}_i(S) = \sum_{s=0}^{k_i} \mathsf{ct}_{i,s} S^s$, with $\mathsf{ct}_{i,s} \in \mathcal{R}_Q$ for $0 \leq i \leq c$. If the ciphertexts $\mathsf{ct}_1, \ldots, \mathsf{ct}_c$ encrypt messages

$\mathsf{m}_1, \ldots, \mathsf{m}_c$ under their respective key bases, the goal is to obtain an affine ciphertext $\mathsf{ct}'(S)$ in the indeterminate S that approximately encrypts the product $\mathsf{m}_1 \cdots \mathsf{m}_c$. Algorithm 3 generalizes operation $\mathsf{Mult}_{\mathsf{evk}}(\mathsf{ct}, \mathsf{ct}')$ from [21] in order to address this problem.

Algorithm 3: General Multiplication with Batched Relinearization

Given an evaluation key $\boldsymbol{U} \in \mathcal{R}_{\tilde{Q}}^{2\times(K-2)d}$ from $(\mathsf{sk}^2, \ldots, \mathsf{sk}^K)$ to sk, generated by Algorithm 1, and c ciphertexts $\mathsf{ct}_1(S), \ldots, \mathsf{ct}_c(S)$ of respective degrees $k_1, \ldots, k_c$ in the secret-key variable S encrypting messages $\mathsf{m}_1, \ldots, \mathsf{m}_c$, all in evaluation form and at the same level l and scale Δ, where $l \geq c-1$ and $k := \sum_{i=1}^{c} k_i \leq K$, this algorithm outputs an affine ciphertext at level $l' := l - c + 1$ in coefficient form that approximately decrypts to $\mathsf{m}_1 \cdots \mathsf{m}_c$ under sk. It uses the precomputed constant $\bar{Q}_{\mathrm{rs}}^{(l,c)} = \prod_{i=0}^{\lceil \log_2 c \rceil - 1} q_{l-i}^{-\lfloor \lceil c/2^i \rceil / 2 \rfloor}$.

1. [Multiply.] Compute the product $a(S) := \prod_{i=1}^{c} \mathsf{ct}_i = \sum_{s=0}^{k} a_s S^s$.
2. [INTT.] Convert $a(S)$ to coefficient form.
3. [Key-switch.] Apply Algorithm 2 with $r_i = a_{i+1}$, $\tilde{b} = 0$ for $1 \leq i < k$; the result is an affine ciphertext $\mathsf{ct}(S) \in \mathcal{R}_Q[S]$.
4. [Add.] Compute $\mathsf{ct}'(S) := a_0 + a_1 S + \mathsf{ct}(S)$.
5. [Rescale.] Output the ciphertext $\mathsf{ct}_{\mathtt{Mult}}(S) = \lfloor \bar{Q}_{\mathrm{rs}}^{(l,c)} \cdot \mathsf{ct}'(S) \rceil$.

Using Karatsuba's method [22] to evaluate the polynomial product in step 1 of Algorithm 3 reduces the number of modular multiplications required considerably. For example, to compute the product $a(S) = a_0 + a_1 S + a_2 S^2 + a_3 S^3 \in \mathcal{R}_Q[S]$ of four affine ciphertexts $\mathsf{ct}_i(S) = c_{i,0} + c_{i,1} S$, we proceed as follows: first calculate $\mathsf{ct}_1(S) \cdot \mathsf{ct}_2(S) = b_0 + b_1 S + b_2 S$ as

$$\begin{aligned} b_0 &= c_{1,0} c_{2,0}, \quad b_2 = c_{1,1} c_{2,1}, \\ b_1 &= (c_{1,1} + c_{1,0})(c_{2,1} + c_{2,0}) - b_0 - b_2, \end{aligned}$$

and do the same for $\mathsf{ct}_3(S) \cdot \mathsf{ct}_4(S) = b'_0 + b'_1 S + b'_2 S$; finally compute

$$a_0 = b_0 b'_0, \quad a_4 = b_2 b'_2, \quad t = b_1 b'_1, \tag{8}$$
$$a_1 = (b_0 + b_1)(b'_0 + b'_1) - a_0 - t, \tag{9}$$
$$a_2 = (b_0 + b_2)(b'_0 + b'_2) - a_0 - a_4 + t, \tag{10}$$
$$a_3 = (b_1 + b_2)(b'_1 + b'_2) - a_4 - t \tag{11}$$

to form the whole polynomial product. The entire process needs 12 multiplications and 17 additions over $\mathcal{R}_Q$. Note that when $c = 2$, previous works have observed that the "grade school" polynomial multiplication method—which in this case performs 4 multiplications and 1 addition, versus 3 and 4—combined with Montgomery arithmetic is actually faster in practice [7].

Since the c input ciphertexts are assumed to have the same scale Δ, their product should have scale Δ^{c-1} prior to rescaling. We take the multiplications to be performed in a balanced-binary-tree fashion, pairing ciphertexts at each level as detailed in Section 4.2. In the equation (8) of the example above, rescaling must remove a factor of Δ^3, which in our batched approach corresponds to dividing by $q_\ell^2 q_{\ell-1}$. This reduces the level of the output ciphertext by 2, which is consistent with the standard relinearization approach.

Theorem 3 shows the correctness of Algorithm 3, and provides a bound on the amount of noise that it introduces.

Theorem 3. *Let $\mathsf{ct}_i(S)$ be an encryption of $\mathsf{m}_i + e_i$ under sk for $1 \le i \le c$, where $\|\mathsf{m}_i\|_\infty < \nu$, $\|e_i\|_\infty < B$. If the conditions in the statement of Theorem 2 hold, then Algorithm 3 returns a ciphertext $\mathsf{ct}_{\mathtt{Mult}}(S)$ such that*

$$\mathsf{ct}_{\mathtt{Mult}}(\mathsf{sk}) \equiv \bar{Q}_{\mathrm{rs}}^{(l,c)} \prod_{i=1}^{c} \mathsf{m}_i + \varepsilon_{\mathrm{br}} \pmod{Q},$$

where $\|\varepsilon_{\mathrm{br}}\|_\infty < \bar{Q}_{\mathrm{rs}}^{(l,c)}((\nu+B)^c - \nu^c + \varepsilon) + \varepsilon_{\mathrm{rs}}^{(l,c)}$, the error term ε is defined in Theorem 2, and $\|\varepsilon_{\mathrm{rs}}^{(l,c)}\|_\infty \le (c-1)/2$.

Proof. Step 1 produces a polynomial $a(S) \in \mathcal{R}_Q[S]$ that, when evaluated at the secret key sk, decrypts to

$$a(\mathsf{sk}) = \prod_{i=1}^{c} \mathsf{ct}_i(\mathsf{sk}) = \prod_{i=1}^{c} (\mathsf{m}_i + e_i) = \prod_{i=1}^{c} \mathsf{m}_i + \sum_{S \subsetneq \{1,\dots,c\}} \left(\prod_{i \in S} \mathsf{m}_i\right)\left(\prod_{i \notin S} e_i\right). \tag{12}$$

Let $e := \sum_{S \subsetneq \{1,\dots,c\}} \left(\prod_{i \in S} \mathsf{m}_i\right)\left(\prod_{i \notin S} e_i\right)$ be the part of the expression above that contains any noise term. Step 3 returns a ciphertext $\mathsf{ct}(S)$ that, by Theorem 2, opens to $\sum_{i=1}^{k-1} r_i \cdot \mathsf{sk}^{i+1} + \varepsilon$ under the secret key sk, where $\|\varepsilon\|_\infty < \frac{1}{2}(1+k+\|\mathsf{sk}\|_1 k)$. In step 4 we define $\mathsf{ct}'(S) = a_0 + a_1 S + \mathsf{ct}(S)$, which opens to

$$\mathsf{ct}'(\mathsf{sk}) = a_0 + a_1 \mathsf{sk} + \mathsf{ct}(\mathsf{sk}) = \sum_{i=0}^{k} a_i \cdot \mathsf{sk}^i + \varepsilon = a(\mathsf{sk}) + \varepsilon$$

as desired. In step 5, we obtain $\mathsf{ct}_{\mathtt{Mult}}(S)$ such that $\mathsf{ct}_{\mathtt{Mult}}(\mathsf{sk}) = \bar{Q}_{\mathrm{rs}}^{(l,c)} \cdot (a(\mathsf{sk}) + \varepsilon) + \varepsilon_{\mathrm{rs}}^{(l,c)}$, where $\varepsilon_{\mathrm{rs}}^{(l,c)}$ is the error of the rescaling, whose l_∞ norm is bounded by $\sum_{i=0}^{\lceil \log 2c \rceil - 1} \lfloor \lceil c/2^i \rceil / 2 \rfloor \le (c-1)/2$. Finally, we analyze the error of batched relinearization by defining it as

$$\varepsilon_{\mathrm{br}} := \mathsf{ct}_{\mathtt{Mult}}(sk) - \bar{Q}_{\mathrm{rs}}^{(l,c)} \prod_{i=1}^{c} \mathsf{m}_i = \bar{Q}_{\mathrm{rs}}^{(l,c)} (a(\mathsf{sk}) + \varepsilon) + \bar{Q}_{\mathrm{rs}}^{(l,c)} - \bar{Q}_{\mathrm{rs}}^{(l,c)} \prod_{i=1}^{c} \mathsf{m}_i,$$

which by Equation (12), equals

$$\varepsilon_{\mathrm{br}} = \bar{Q}_{\mathrm{rs}}^{(l,c)} \left(\sum_{S \subsetneq \{1,\dots,c\}} \left(\prod_{i \in S} \mathsf{m}_i\right)\left(\prod_{i \notin S} e_i\right) + \varepsilon \right) + \varepsilon_{\mathrm{rs}}^{(l,c)}.$$

Finally, use the bounds $\|\mathsf{m}_i\|_\infty < \nu$ and $\|e_i\|_\infty < B$ to show

$$\|\varepsilon_{\mathrm{br}}\|_\infty < \bar{Q}_{\mathrm{rs}}^{(l,c)}((\nu+B)^c - \nu^c + \varepsilon) + \varepsilon_{\mathrm{rs}}^{(l,c)}. \qquad \square$$

4.2 Noise Accumulation Comparison

We compare noise accumulation under the standard binary-tree-like and the proposed batched approaches for multiplying c affine ciphertexts. We first present concrete error bounds for the case $c = 4$; next, we provide asymptotic bounds for larger c values.

Standard Relinearization for $c = 4$. The conventional approach to multiply $c = 2^h$ affine ciphertexts is to organize the computation as a binary tree. Denote the ciphertexts whose product is to be found by $\mathsf{ct}_{h,0}(S), \ldots, \mathsf{ct}_{h,c-1}(S)$, and let l be their initial level. Formally, for $k = h-1, h-2, \ldots, 0$, set $\mathsf{ct}_{k,i}(S)$ to the output of Algorithm 3 on inputs $\mathsf{ct}_{k+1,2i}(S)$ and $\mathsf{ct}_{k+1,2i+1}(S)$), for each $0 \le i < 2^k$. The result appears in $\mathsf{ct}_{0,0}(S)$, which is at level $l - h$ due to the rescales at each level of the tree. This procedure performs $\sum_{k=0}^{h-1} 2^k = h - 1$ key switches from sk^2 to sk, and it is amenable to parallelization.

Let us analyze the noise growth incurred by this approach in the case $c = 2^2$. Each input ciphertext $\mathsf{ct}_{2,i}(S)$ opens to $\mathsf{m}_i + e_i$, where $\|\mathsf{m}_i\|_\infty \le \nu$ and $\|e_i\|_\infty \le B$. At level $k = 1$, we construct $\mathsf{ct}_{1,0}(S)$ and $\mathsf{ct}_{1,1}(S)$, which Theorem 3 shows are encryptions of $q_l^{-1} \cdot \mathsf{m}_0 \cdot \mathsf{m}_1$ and $q_l^{-1} \cdot \mathsf{m}_2 \cdot \mathsf{m}_3$ plus error terms bounded by $q_l^{-1}(2B\nu + B^2 + \varepsilon) + \varepsilon_{\mathrm{rs}}^{(l,2)}$. The new message and error bounds are $\nu_1 := \nu^2/q_l$ and $B_1 := (2B\nu + B^2 + \varepsilon)/q_l + \varepsilon_{\mathrm{rs}}^{(l,2)}$. At level $k = 0$, we construct $\mathsf{ct}_{0,0}(S)$ as the product of $\mathsf{ct}_{1,0}(S)$ and $\mathsf{ct}_{1,1}(S)$. This final ciphertext opens to the desired (rescaled) product $q_l^{-1}q_{l-1}^{-1} \cdot \mathsf{m}_0 \cdot \mathsf{m}_1 \cdot \mathsf{m}_2 \cdot \mathsf{m}_3$ plus an error term bounded by

$$\begin{aligned} B_0 &= q_{l-1}^{-1}(2B_1\nu_0 + B_1^2 + \varepsilon) + \varepsilon_{\mathrm{rs}}^{(l-1,2)} \\ &= 2q_{l-1}^{-1}\left(\frac{2B\nu + B^2 + \varepsilon}{q_l} + \varepsilon_{\mathrm{rs}}^{(l,2)}\right)\frac{\nu^2}{q_l} \\ &\quad + q_{l-1}^{-1}\left(\frac{2B\nu + B^2 + \varepsilon}{q_l} + \varepsilon_{\mathrm{rs}}^{(l,2)}\right)^2 + q_{l-1}^{-1}\varepsilon + \varepsilon_{\mathrm{rs}}^{(l-1,2)}. \end{aligned} \tag{13}$$

Batched Relinearization for $c = 4$. Theorem 3 states that using Algorithm 3 to compute the same product of four ciphertexts yields an approximate encryption of $q_l^{-2}q_{l-1}^{-1} \cdot \mathsf{m}_0 \cdot \mathsf{m}_1 \cdot \mathsf{m}_2 \cdot \mathsf{m}_3$ with noise bounded by

$$\frac{(\nu+B)^4 - \nu^4 + \varepsilon}{q_l^2 \cdot q_{l-1}} + \varepsilon_{\mathrm{rs}}^{(l,4)} = \frac{4\nu^3 B + 6\nu^2 B^2 + 4\nu B^3 + B^4 + \varepsilon}{q_l^2 \cdot q_{l-1}} + \varepsilon_{\mathrm{rs}}^{(l,4)}. \tag{14}$$

Comparison for $c = 4$. Observe that the dominant term in the noise bounds (13) and (14) is $4\nu^3 B/q_l^2 q_{l-1}$ in both cases. Importantly, however, the standard method introduces a large number of noise terms of lower order depending on ν, B, ε,

and $\varepsilon_{\mathrm{rs}}$'s. In the batched case the number of these terms is smaller; and $\varepsilon_{\mathrm{rs}}^{(l,4)}$ is added only at the end, remaining linear and not multiplied by any of the other bounds. This agrees with the results of Akherati *et al.* [1] for $c = 3$.

Scaling to Larger c. In the binary-tree-based multiplication method (where we assume $c = 2^h$ for simplicity), the message and the error bounds at each level $k = h-1, h-2, \ldots, 0$ are given by the two-variable recurrence

$$\nu_k = \frac{\nu_{k+1}^2}{q_{l+k+1-h}}, \qquad B_k = \frac{2B_{k+1}\nu_{k+1} + B_{k+1}^2 + \varepsilon}{q_{l+k+1-h}} + \varepsilon_{\mathrm{rs}}^{(l+k+1-h,2)},$$

with initial conditions $(\nu_h, B_h) = (\nu, B)$, where l is the level of all input ciphertexts. By contrast, Theorem 3 states that the batched relinearization technique introduces an error no greater than $B_c = \bar{Q}_{\mathrm{rs}}^{(l,c)} \cdot ((\nu + B)^c - \nu^c + \varepsilon) + \varepsilon_{\mathrm{rs}}^{(l,c)}$ after the same number of multiplications; the message bound is $\nu_c = \bar{Q}_{\mathrm{rs}}^{(l,c)} \cdot \nu^c$. We see that the dominant term in both error bounds is the same, namely the one of order $\nu^{c-1}B$. But again we see that the batched relinearization approach introduces the rounding error in the lowering step at the end, whereas the standard relinearization approach introduces crossed terms involving the rounding error and its square at every level of the binary tree.

4.3 Homomorphic Evaluation of Approximation Polynomials

The General Batched Relinearization technique, described in Algorithm 3, has clear application in the evaluation of approximation polynomials for nonlinear functions, which is needed in the context of privacy-preserving deep learning. We discuss three concrete instantiations of our algorithm to three different evaluations that benefit from the batching technique.

Rational Approximation using Goldschmidt's Algorithm.

Goldschmidt's algorithm [18] is frequently used to approximate the reciprocal of a number. For example, Lee *et al.* [24] and Moon *et al.* [28] employ it to evaluate the denominator of the softmax function. As an example, we consider the approximation of the reciprocal of a real positive quantity discussed by Chen *et al.* [11]:

$$\frac{1}{X} \approx \prod_{i=0}^{d} \left(1 + (1-X)^{2^i}\right).$$

Let us take $d = 2$ to illustrate the basic idea. Given a ciphertext $\mathsf{ct}(S)$ of degree k in S,

Compute $\mathsf{ct}_1(S) = \mathsf{pt}_1 - \mathsf{ct}(S)$, where pt_1 is an encoding of the all-ones vector.
Compute $\mathsf{ct}_2(S) = \mathsf{ct}_1(S) \cdot \mathsf{ct}_1(S)$ and $\mathsf{ct}_4(S) = \mathsf{ct}_2(S) \cdot \mathsf{ct}_2(S)$,
  which are of respective degrees $2k$ and $4k$.
Compute $\mathsf{ct}'(S) = (\mathsf{pt}_1 + \mathsf{ct}_1(S)) \cdot (\mathsf{pt}_1 + \mathsf{ct}_2(S)) \cdot (\mathsf{pt}_1 + \mathsf{ct}_4(S))$ of degree $7k$.
Execute steps 2 through 5 of Algorithm 3 with $a(S) = \mathsf{ct}'(S)$;
if $\mathsf{ct}(S)$ opens to $\mathsf{m} + e$, then the result $\mathsf{ct}_{\mathsf{gs}}(S)$ opens to
  $(1 + (1 - \mathsf{m} - e))(1 + (1 - \mathsf{m} - e)^2)(1 + (1 - \mathsf{m} - e)^4)$.

In this process, we accumulate the results of all multiplications in $\mathsf{ct}'(S)$, and then relinearize this polynomial of degree $7k$ in the secret-key indeterminate S into an affine one $\mathsf{ct}_{\mathsf{gs}}(S)$. It is possible to continue accumulating multiplications using the new ciphertext $\mathsf{ct}_{\mathsf{gs}}(S)$ up to another threshold d', and then perform the batched relinearization procedure again. This method requires storing a larger amount of data in memory—notice that $\mathsf{ct}'(S)$ alone has $7kN$ integer coefficients—but in compensation we reduce the number of relinearization steps from four to just one, i.e., we avoid six modulus lowerings.

Exponentiation Approximation. The exponential function is approximated in Lu *et al.* [26] and Zhang *et al.* [35] as

$$\exp(X) \approx \left(1 + \frac{X}{2^r}\right)^{2^r}.$$

We describe a possible sequence of steps to compute this approximation up to order $r = 3$. Given a ciphertext $\mathsf{ct}(S)$ of degree k in S,

Compute $\mathsf{ct}_1(S) = (1 + \mathsf{ct}(S)/2^r)$ of degree k.
Compute $\mathsf{ct}_2(S) = \mathsf{ct}_1(S) \cdot \mathsf{ct}_1(S)$ of degree $2k$.
Compute $\mathsf{ct}_4(S) = \mathsf{ct}_2(S) \cdot \mathsf{ct}_2(S)$ of degree $4k$.
Compute $\mathsf{ct}_8(S) = \mathsf{ct}_4(S) \cdot \mathsf{ct}_4(S)$ of degree $8k$.
Execute steps 2–5 of Algorithm 3 with $a(S) = \mathsf{ct}_8(S)$.

This procedure is a typical example of repeated squaring. Under batched relinearization, the number of necessary relinearizations and modulus lowerings drops from 3 to 2 and from 6 to 2, respectively, but the large number of polynomial multiplications starts to be considerable. Of course, batched relinearization may be performed at an earlier point of the computation. Practitioners should judge where the tradeoff between more expensive key-switching and other operations lies in each particular application.

GELU Approximation. Transformer architectures commonly employ the GELU activation function,

$$\mathrm{GELU}(X) = \frac{X}{2}\left(1 + \mathrm{erf}\left(\frac{X}{\sqrt{2}}\right)\right) \approx \frac{X}{2}\left(1 + \tanh\left(\sqrt{\frac{2}{\pi}(X + cX^3)}\right)\right).$$

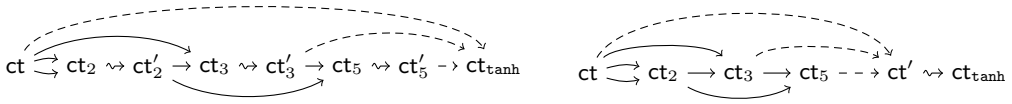
Here $\mathrm{erf}(X)$ is the Gaussian error function, which some deep-learning libraries approximate using the hyperbolic tangent function $\tanh(X)$ as shown above. Approximation strategies focus either on approximating only the $\mathrm{erf}(X)$ and $\tanh(X)$ functions or the entire GELU function.

The Taylor polynomials of $\tanh(X)$ are odd functions whose homomorphic evaluation can benefit from batched relinearization. Consider, for example, the degree-5 approximation $\tanh(X) \approx c_1X + c_3X^3 + c_5X^5$. Given a ciphertext $\mathsf{ct}(S)$

of degree k in S,

Compute $\mathsf{ct}_2(S) = \mathsf{ct}(S) \cdot \mathsf{ct}(S)$ of degree $2k$.
Compute $\mathsf{ct}_3(S) = \mathsf{ct}(S) \cdot \mathsf{ct}_2(S)$ of degree $3k$.
Compute $\mathsf{ct}_5(S) = \mathsf{ct}_3(S) \cdot \mathsf{ct}_2(S)$ of degree $5k$.
Compute $\mathsf{ct}'(S) = c_1\mathsf{ct}(S) + c_3\mathsf{ct}_3(S) + c_5\mathsf{ct}_5(S)$ of degree $5k$.
Perform steps 2 through 5 of Algorithm 3 with $a(S) = \mathsf{ct}'(S)$;
if $\mathsf{ct}(S)$ opens to $\mathsf{m} + e$, the output $\mathsf{ct}_{\mathtt{tanh}}(S)$ is an affine ciphertext that approximately opens to $c_1(\mathsf{m} + e) + c_3(\mathsf{m} + e)^3 + c_5(\mathsf{m} + e)^5$.

Similar to the previous examples, batched relinearization allows us to decrease the number of modulus lowerings from $4 \cdot 2 = 8$ to 2. Figure 1 compares the original and the batched procedures for this example.



(a) The conventional approach.

(b) Our batched approach.

Fig. 1: Two ways to homomorphically approximate $\tanh(X)$ with a degree-5 polynomial, given an encryption ct of X. Solid, wiggly, and dashed arcs represent respectively multiplications, relinearizations, and additions.

5 Double-Hoisted Matrix–Vector Multiplication

As another example of the application of the batched key-switching technique to a practical problem, we consider the homomorphic evaluation of a matrix–vector product $\boldsymbol{M}\boldsymbol{v}$, where $\boldsymbol{M} \in \mathbb{C}^{n \times n}$ is a cleartext matrix and $\boldsymbol{v} \in \mathbb{C}^n$ is the slot vector associated with a CKKS ciphertext. This operation arises frequently in practice, since, for example, the coefficient-to-slots and slot-to-coefficients steps in a CKKS bootstrapping circuit (which raises the level of a ciphertext to allow further computation) are linear transformations [8]. Also, recently-emerged FHE inference engines tend to express the linear layers of a DNN as matrix–vector products [14, 30].

Halevi-Shoup [19] observed that the product $\boldsymbol{M}\boldsymbol{v}$ may be expressed in terms of the *generalized diagonals* of $\boldsymbol{M}$, namely all vectors $\left[(m_{i,i+k \pmod n})_{0 \le i < n}\right]$ for $0 \le k < n$, where the column indices wrap around modulo n. Let $m_0, \ldots, m_{n-1} \in \mathcal{R}$ be plaintexts encoding these diagonals. In this way, the formula $\sum_{k=0}^{n-1} m_k \cdot \mathsf{Rotate}(\mathsf{ct}(S), k)$ converts an encryption $\mathsf{ct}(S)$ of $\boldsymbol{v}$ into one of $\boldsymbol{M}\boldsymbol{v}$. The operation $\mathsf{Rotate}(\mathsf{ct}(S), k)$ applies the automorphism $\phi_k \colon X \to X^{5^k} \pmod{X^N + 1}$ to a ciphertext $\mathsf{ct}(S)$, which corresponds to a circular shift of its underlying cleartext vector by k positions to the left. Afterwards it uses key-switching in order to

obtain a valid encryption under the original secret key sk rather than under $\phi_k(\mathsf{sk})$. This last step makes the computation of the rotations the most computationally costly part in the homomorphic evaluation of $\boldsymbol{Mv}$. Fortunately, the number of rotations can be brought down from $\Theta(n)$ to $\Theta(\sqrt{n})$ by considering a *baby-step giant-step* (BSGS) decomposition [20]: write $n = n_\mathrm{b} n_\mathrm{g}$ for some integers n_b and n_g, construct the ciphertexts $\mathsf{ct}_i(S) := \mathsf{Rotate}(\mathsf{ct}(S), i)$ for $0 \leq i < n_\mathrm{b}$, and compute

$$\sum_{0 \leq j < n_\mathrm{g}} \phi_{n_\mathrm{b} j} \left(\sum_{0 \leq i < n_\mathrm{b}} \phi_{-n_\mathrm{b} j}(m_{n_\mathrm{b} j + i}) \cdot \mathsf{ct}_i(S) \right). \tag{15}$$

This procedure happens to be a good candidate for the hoisting optimization discussed in Section 3: the n_b rotations of $\mathsf{ct}(S)$ may be constructed in batch from the same gadget decomposition of $\mathsf{ct}(S)$, thereby reducing the number of NTT operations and RNS reconstructions involved. Bossuat *et al.* [7] realized that hoisting may be applied a second time, by delaying the lowering of the rotations to $\mathcal{R}_Q$ until these are grouped into each term of the outer sum in Equation (15). With this method, the total number of modulus lowerings goes down from $n + n_\mathrm{g}$ to $n_\mathrm{g} + 1$, which saves time in spite of carrying the plaintext-ciphertext multiplications over the extended ring $\mathcal{R}_{\tilde{Q}}$.

Bossuat *et al.*'s approach is the current state of the art for evaluating dense matrix–vector products homomorphically, and it is part of the bootstrapping circuits provided by popular CKKS libraries such as Lattigo [15] and OpenFHE [3]. Although its execution time and space requirements have been analyzed in detail (both theoretically and empirically), its precise impact on noise growth remains unknown. In order to investigate this question, we shall now look into the "baby" and "giant" stages of the double-hoisted algorithm more deeply. (The results in Section 3 will be relevant when deriving upper bounds on the accumulated noise after each stage.) Among other things, this will let us to confirm the rule of thumb that the noise introduced by the coefficient-to-slots and slot-to-coefficients steps in the aforementioned bootstrap circuits is negligible.

5.1 Batched Rotations

Under the generalized-diagonals formulation, the first step in applying a linear transformation to an encryption $\mathsf{ct}(S)$ of a vector is to compute the n_b distinct rotations of $\mathsf{ct}(S)$ that appear in the sum (15). We have already seen that this task is amenable to hoisting, and in fact it almost follows the structure of Program (4). (For reasons to be explained shortly, the "main" section of the double-hoisted algorithm introduces less noise when it works with the rotations directly over the extended ring $\mathcal{R}_{\tilde{Q}}$. So here we insist that the output of the "baby" part is not lowered to $\mathcal{R}_Q$ prematurely, i.e., we skip the final modulus lowering step.) Indeed, extracting the initial gadget step shared by all the key-switches—just as we did to obtain Program (5)—yields Algorithm 4.

Again, the resulting rotations (together with $\tilde{\mathsf{ct}}_0(S)$) live in the extended ring $\mathcal{R}_{\tilde{Q}}[S]$; if necessary, one may lower them to $\mathcal{R}_Q[S]$ by switching $\tilde{\mathsf{ct}}_i(S)$ to

Algorithm 4: Hoisted Automorphism Applications [7]

Let $\rho_1, \ldots, \rho_n$ be automorphisms of $\mathcal{R}$, and let ρ_0 be the identity map. Given a ciphertext $\mathsf{ct}(S) = b + aS \in \mathcal{R}_Q[S]$ in coefficient form, and an evaluation key $\boldsymbol{U} \in \mathcal{R}_{\tilde{Q}}^{2\times nd}$ from $(\rho_1(\mathsf{sk}), \ldots, \rho_n(\mathsf{sk}))$ to sk produced by Algorithm 1 for a gadget vector $\boldsymbol{g} \in \mathbb{Z}_{\tilde{Q}}^d$, this algorithm outputs an affine ciphertext $\tilde{\mathsf{ct}}_i(S) \in \mathcal{R}_{\tilde{Q}}[S]$ in evaluation form that approximately decrypts to $P \cdot \rho_i(\mathsf{ct}(\mathsf{sk}))$ under sk, for each $0 \le i \le n$. It uses a precomputed matrix $\boldsymbol{U}'$ with entries $u'_{j,k} = \rho^{-1}_{\lfloor (k-1)/d \rfloor + 1}(u_{j,k})$, for $j \in \{0, 1\}$ and $1 \le k \le nd$.

1. [Decompose.] Compute the gadget decomposition $\boldsymbol{d} \in \mathcal{R}_{\tilde{Q}}^d$ of a with respect to the basis $\boldsymbol{g}$.
2. [Extend.] Find the RNS representation $\tilde{\boldsymbol{d}}$ of $\boldsymbol{d}$ over $\mathcal{R}_{\tilde{Q}}^d$. Perform the same for $P \cdot \mathsf{ct}(S) \in \mathcal{R}_Q[S]$ in order to obtain $\tilde{\mathsf{ct}}_0(S) = \tilde{b} + \tilde{a}S \in \mathcal{R}_{\tilde{Q}}[S]$.
3. [NTT.] Convert $\tilde{\boldsymbol{d}}$ and $\tilde{\mathsf{ct}}_0(S)$ to evaluation form.
4. [Key-switch.] Compute the row vector
$$\left[\tilde{\mathsf{ct}}_1(S) \cdots \tilde{\mathsf{ct}}_n(S)\right] := \left[1 \; S\right] \boldsymbol{U}(\boldsymbol{I}_n \otimes \tilde{\boldsymbol{d}}) = \left[\left((\boldsymbol{u}_{0,i} + \boldsymbol{u}_{1,i}S)\tilde{\boldsymbol{d}}\right)_i\right].$$
Here $\boldsymbol{I}_n$ is the identity matrix of order n, and $\otimes$ denotes the Kronecker product.
5. [Rotate.] For $1 \le i \le n$, set $\tilde{\mathsf{ct}}_i(S)$ to $\rho_i\left(\tilde{b} + \tilde{\mathsf{ct}}_i(S)\right)$.

coefficient form and finding the RNS representation of $\lfloor P^{-1} \cdot \tilde{\mathsf{ct}}_i(S) \rceil$ over $\mathcal{R}_Q[S]$ via the fast basis conversion algorithm. This gives a general method for batching automorphism applications, but it does not apply to the problem at hand.

One interesting aspect of Algorithm 4 is that its step 4, which is in charge of key-switching, constructs ciphertexts that decrypt under the secret keys $\rho_i^{-1}(\mathsf{sk})$ rather than the original sk. Bossuat *et al.* [7] observed that this change from the traditional method of key-switching $\rho_i(a)$ allows us to apply ρ_i only at the final step of the algorithm, thereby reducing the total number of automorphism applications by a factor of 3/2. We prove that the algorithm (and this change) is correct, and give an explicit bound for the noise growth it incurs in Theorem 4.

Theorem 4. *Let $Q = q_0 \cdots q_l$ be a product of coprime integers, let $Q = D_1 \cdots D_d$ be the digit decomposition used to construct a gadget vector $\boldsymbol{g} \in \mathbb{Z}_Q^d$, and let ε_{f} be an upper bound on the ℓ_∞ norm of the error term in a fresh encryption. Each ciphertext $\tilde{\mathsf{ct}}_i(S)$ produced by Algorithm 4 opens to $\tilde{\mathsf{ct}}_i(\mathsf{sk}) \equiv P \cdot \rho_i(\mathsf{ct}(\mathsf{sk})) + \varepsilon \pmod{Q}$, where $\|\varepsilon\|_\infty \le \frac{1}{2} N \varepsilon_{\mathrm{f}} (l+1) \max_{1 \le j \le d} D_j$.*

Proof. After precomputation, the ith block column $\boldsymbol{U}'_i = \begin{bmatrix} \boldsymbol{u}'^T_{0,i} & \boldsymbol{u}'^T_{1,i} \end{bmatrix}^T \in \mathcal{R}_{\tilde{Q}}^{2\times d}$ of $\boldsymbol{U}'$, $1 \le i \le n$, stops being an evaluation key from $\rho_i(\mathsf{sk})$ to sk and becomes an evaluation key from sk to $\rho_i^{-1}(\mathsf{sk})$. To see why, notice that $\boldsymbol{u}_{1,i}$ and $\boldsymbol{u}_{0,i}$ are

replaced respectively by $\boldsymbol{u}'_{1,i} = \rho_i^{-1}(\boldsymbol{u}_{1,i})$ and

$$\begin{aligned}\rho_i^{-1}(\boldsymbol{u}_{0,i}) &= -\rho_i^{-1}(\mathsf{sk}) \cdot \rho_i^{-1}(\boldsymbol{u}_{1,i}) + P \cdot \rho_i^{-1}\left(\rho_i(\mathsf{sk})\right) \cdot \boldsymbol{g}^T + \rho_i^{-1}(\boldsymbol{e}_i) \mod \tilde{Q} \\ &= -\rho_i^{-1}(\mathsf{sk}) \cdot \boldsymbol{u}'_{1,i} + P \cdot \mathsf{sk} \cdot \boldsymbol{g}^T + \boldsymbol{e}'_i \mod \tilde{Q}\end{aligned}$$

in the notation of Algorithm 1, because an automorphism distributes over addition and multiplication. Here and in the sequel we take a ring automorphism to apply entrywise to matrices and vectors of ring elements. Importantly, the corresponding entries of the terms $\boldsymbol{e}_i$ and $\boldsymbol{e}'_i = \rho_i^{-1}(\boldsymbol{e}_i)$ have the same ℓ_∞ norm, because an automorphism of $\mathcal{R}$ permutes the coefficients of its input without changing their absolute value—doing arithmetic modulo the negacyclic polynomial $X^N + 1$ may at most induce a sign change in the coefficients.

A similar argument shows that the algorithm terminates with

$$\tilde{\mathsf{ct}}_i(S) \equiv \rho_i(\tilde{b} + \left[1\ S\right] \boldsymbol{U}_i \tilde{\boldsymbol{d}}) = \rho_i(\tilde{b}) + \left[1\ S\right] \rho_i(\boldsymbol{U}_i)\rho_i(\tilde{\boldsymbol{d}}) \pmod{Q}.$$

It is not difficult to see that $\rho_i(\tilde{\boldsymbol{d}})$ is the gadget decomposition of $\rho_i(a)$ in the basis $\boldsymbol{g}$. This observation, together with the fact that $\tilde{b}$ is the extension of $P \cdot b$ to $\mathcal{R}_{\tilde{Q}}$, implies that $\tilde{\mathsf{ct}}_i(\mathsf{sk}) = \rho_i(\tilde{b}) + \left(P \cdot \rho_i(\mathsf{sk}) \cdot \boldsymbol{g}^T + \boldsymbol{e}_i\right) \rho_i(\tilde{\boldsymbol{d}}) \bmod Q$ is approximately equal to $P \cdot \rho_i(\mathsf{ct}(\mathsf{sk})) = P \cdot \rho_i(b + a \cdot \mathsf{sk})$ as desired. Continue as in the proof of Theorem 2 to bound the error $\left|\!\left|\!\left|\varepsilon(\mathsf{sk})\right|\!\right|\!\right|_\infty = \left|\!\left|\!\left|\boldsymbol{e}_i \rho_i(\tilde{\boldsymbol{d}})\right|\!\right|\!\right|_\infty$. □

5.2 The Baby and Giant Steps

We are now ready to analyze the double-hoisted BSGS algorithm in its entirety. All the rotations of vector $\boldsymbol{v}$ needed to evaluate the linear transformation (15) may be obtained by invoking Algorithm 4 with $n = n_\mathrm{b} - 1$ and the automorphisms $\phi_1, \ldots, \phi_{n_\mathrm{b}-1}$, corresponding to rotations by $1, 2, \ldots, n_\mathrm{b} - 1$ slots, respectively. The remaining job in forming an encryption of $\boldsymbol{M}\boldsymbol{v}$ is:

(i) to multiply these rotations together with the original unrotated input by each group of n_b consecutive diagonals of the plaintext matrix $\boldsymbol{M}$; and
(ii) to add the resulting n_g ciphertexts after being rotated by multiples of n_b slots.

Before we formalize the entire procedure, let us give some comments about these two individual steps.

First, it is worth noting that the multiplications in step (i) are carried out over the extended ring $\mathcal{R}_{\tilde{Q}}$. The alternative—as we discussed in the context of Algorithm 4—would be to lower the n_b rotations involved from $\mathcal{R}_{\tilde{Q}}$ to $\mathcal{R}_Q$, and then go on to step (i). Bossuat *et al.* [7] argued that, since the additional cost of the extended multiplications is usually smaller than that of the extra modulus lowerings, it is best to wait until step (i) is complete before going down to $\mathcal{R}_Q$. (Of course, this is no advantage at all when n_g is considerably larger than n_b, because in that case we would be doing *more* reductions. It is customary to set the parameters n_b and n_g such that $8 \leq n_\mathrm{b}/n_\mathrm{g} \leq 16$, for this and other reasons explained in the cited paper.) Working over $\mathcal{R}_{\tilde{Q}}$ is also helpful from the

standpoint of noise growth, since this ring provides extra precision and allows for a later modulus lowering operation to effectively reduce the accumulated error by a factor of P. The results presented below quantify the influence of this strategy on the total amount of noise introduced by the double-hoisted method.

Step (ii) is another example of a circuit that combines the results of several key-switches into a single ciphertext, and thus a viable candidate for acceleration via the batched Algorithm 2 from Section 3. Note that in this case the source keys are the results of applying the automorphisms $\phi_{n_\mathrm{b}}, \phi_{2n_\mathrm{b}}, \ldots, \phi_{(n_\mathrm{g}-1)n_\mathrm{b}}$ to the destination secret key sk.

We are now in a position to describe the procedure more formally in Algorithm 5. An alert reader will have noticed that step 5 of Algorithm 5 is slightly

Algorithm 5: Double-Hoisted BSGS Matrix–Vector Multiplication [7]

Let $\tilde{m}_0, \ldots, \tilde{m}_{n-1} \in \mathcal{R}_{\tilde{Q}}$ be plaintexts encoding the generalized diagonals of a matrix $\boldsymbol{M} \in \mathbb{C}^{n \times n}$. Given evaluation keys $\boldsymbol{U}_\mathrm{b}$ from $(\phi_1(\mathsf{sk}), \ldots, \phi_{n_\mathrm{b}-1}(\mathsf{sk}))$ to sk, and $\boldsymbol{U}_\mathrm{g}$ from $(\phi_{n_\mathrm{b}}(\mathsf{sk}), \phi_{2n_\mathrm{b}}(\mathsf{sk}), \ldots, \phi_{(n_\mathrm{g}-1)n_\mathrm{b}}(\mathsf{sk}))$ to sk constructed by Algorithm 1 for a gadget basis $\boldsymbol{g} \in \mathbb{Z}_Q^d$, where n_b and n_g are integers such that $n = n_\mathrm{b} n_\mathrm{g}$, this algorithm turns an affine encryption $\mathsf{ct}(S) \in \mathcal{R}_Q[S]$ of a vector $\boldsymbol{v} \in \mathbb{C}^n$ under sk into an encryption $\mathsf{ct}'(S) \in \mathcal{R}_Q[S]$ of $\boldsymbol{M}\boldsymbol{v}$ under sk. Both $\mathsf{ct}(S)$ and $\mathsf{ct}'(S)$ are in coefficient form; the $\tilde{m}_k$'s are in evaluation form.

1. [Rotate input.] Apply Algorithm 4 with $\boldsymbol{U} = \boldsymbol{U}_\mathrm{b}$ to obtain the rotation $\tilde{\mathsf{ct}}_i(S) \in \mathcal{R}_{\tilde{Q}}[S]$ of $\mathsf{ct}(S)$ by i slots, for all $0 \le i < n_\mathrm{b}$.
2. [Baby steps.] For $0 \le j < n_\mathrm{g}$, set $\tilde{\mathsf{ct}}'_j(S)$ to $\sum_{i=0}^{n_\mathrm{b}-1} \phi_{-n_\mathrm{b} j}(\tilde{m}_{n_\mathrm{b} j + i}) \cdot \tilde{\mathsf{ct}}_i(S)$.
3. [INTT.] Convert $\tilde{\mathsf{ct}}'_j(S)$ to coefficient form, for $0 \le j < n_\mathrm{g}$.
4. [Lower.] Set a_j to $\lfloor P^{-1} \cdot \tilde{a}_j \rceil$ for each $0 \le j < n_\mathrm{g}$, where $\tilde{\mathsf{ct}}'_j(S) = \tilde{b}_j + \tilde{a}_j S$.
5. [Giant steps.] Apply Algorithm 2 with $r_j = \phi_{n_\mathrm{b} j}(a_j)$ for $1 \le j < n_\mathrm{g}$, $\tilde{b} = \sum_{j=1}^{n_\mathrm{g}-1} \phi_{n_\mathrm{b} j}(\tilde{b}_j)$, and $\boldsymbol{U} = \boldsymbol{U}_\mathrm{g}$; add $\tilde{\mathsf{ct}}'_0(S)$ to the result $\mathsf{ct}'(S) \in \mathcal{R}_Q[S]$.
6. [Rescale.] Output the ciphertext $\mathsf{Rescale}(\mathsf{ct}'(S))$.

different from its original description by Bossuat *et al.* [7, lines 15 and 16 of Algorithm 6]. As in a regular CKKS rotation, we apply each "giant" automorphism $\phi_{n_\mathrm{b} j}$ to its corresponding polynomial a_j before constructing its gadget decomposition; the original method delays the application of the automorphism until the key-switch of a_j from sk to $\phi_{n_\mathrm{b} j}^{-1}(\mathsf{sk})$ is complete. The reason for preferring the latter strategy when computing the baby rotations (step 1) is that it allows us to generate all the rotations from a single gadget decomposition. But in this case the ciphertext $\tilde{\mathsf{ct}}'_j(S)$ only needs to be rotated once, so it is faster to apply $\phi_{n_\mathrm{b} j}$ to a_j over $\mathcal{R}_Q$ than to the linear part of the key-switched ciphertext over the extended $\mathcal{R}_{\tilde{Q}}$. In other words, "absorbing" the automorphisms into the evaluation key $\boldsymbol{U}_\mathrm{g}$ actually hurts performance. Fortunately, the proof of Theorem 4 shows that this change poses no effect on noise; hence Theorem 5 remains valid for the original version of the algorithm.

Theorem 5. *Let $Q = q_0 \cdots q_{l-1}$ and $P = p_1 \cdots p_k$ be products of coprime integers, and let $Q = D_1 \cdots D_d$ be the digit decomposition used to construct a gadget vector $\boldsymbol{g} \in \mathbb{Z}_Q^d$. Define $C := \frac{1}{2} N \varepsilon_{\mathrm{f}} (l+1) \max_{1 \le j \le d} D_j$, where ε_{f} is an upper bound on the ℓ_∞ norm of a fresh encryption's error. Also let $\mathsf{ct}(S) \in \mathcal{R}_Q[S]$ be a ciphertext that opens to $\mathsf{ct}(\mathsf{sk}) \equiv \mathsf{m} + \varepsilon \pmod{Q}$. If $P > C$, Algorithm 5 outputs a ciphertext that decrypts to $\sum_{k=0}^{n-1} \tilde{m}_k \cdot \phi_k(\mathsf{m}) + \varepsilon'(\mathsf{sk})$, where the error term satisfies*

$$\|\varepsilon'(\mathsf{sk})\|_\infty \le \frac{1}{q_l} \left(n\nu B + \frac{k\left((n_{\mathrm{g}}+1)\|\mathsf{sk}\|_1 + 1\right) + 1}{2} \right) + B_{\mathrm{rs}}, \tag{16}$$

for some $\nu, B \ge 0$ such that $|||\tilde{m}_i|||_\infty \le \nu$ and $|||\varepsilon|||_\infty \le B$ for all $0 \le i < n$.

Proof. By Minkowski's inequality and Theorem 4, after step 1 we have $\tilde{\mathsf{ct}}_i(\mathsf{sk}) \equiv P \cdot \phi_i(\mathsf{m}) + \varepsilon_1 \pmod{Q}$ for some error $\varepsilon_1 \in \mathcal{R}$ such that $|||\varepsilon_1|||_\infty \le P \cdot |||\varepsilon|||_\infty + C \le P \cdot B + C$. Next, step 2 computes a linear combination of n_{b} of these ciphertexts with plaintext diagonals of matrix $\boldsymbol{M}$. Each of the resulting ciphertexts decrypts to

$$\tilde{\mathsf{ct}}'_j(\mathsf{sk}) = P \cdot \sum_{i=0}^{n_{\mathrm{b}}} \phi_{-n_{\mathrm{b}} j}(\tilde{m}_{n_{\mathrm{b}} j + i}) \cdot \phi_i(\mathsf{m}) + \varepsilon_2, \tag{17}$$

where the error term satisfies

$$\|\varepsilon_2\|_\infty \le |||\varepsilon_2|||_\infty \le \sum_{i=0}^{n_{\mathrm{b}}} |||\phi_{-n_{\mathrm{b}} j}(\tilde{m}_{n_{\mathrm{b}} j + i})|||_\infty |||\varepsilon_1|||_\infty \le n_{\mathrm{b}} \nu |||\varepsilon_1|||_\infty$$

due to the fact that N is a power of 2 and all the necessary products are carried out in the evaluation domain. The approximate modulus lowering of $\tilde{a}_j$ from $\mathcal{R}_{\tilde{Q}}$ to $\mathcal{R}_Q$ in step 4 yields a polynomial $a_j = P^{-1} \cdot \tilde{a}_j + \varepsilon_4$ with $\|\varepsilon_4\|_\infty \le k/2$. (See Appendix A of [9].) Finally, by Theorem 2, step 5 yields a ciphertext $\mathsf{ct}'(S)$ that opens to

$$\mathsf{ct}'(\mathsf{sk}) = \sum_{j=0}^{n_{\mathrm{g}}} \phi_{n_{\mathrm{b}} j}(a_j) \cdot \phi_{n_{\mathrm{b}} j}(\mathsf{sk}) + P^{-1} \sum_{j=0}^{n_{\mathrm{g}}} \phi_{n_{\mathrm{b}} j}(\tilde{b}_j) + \varepsilon_5,$$

with $\|\varepsilon_5\|_\infty \le (1 + k + \|\mathsf{sk}\|_1 k)/2$. We can rewrite this expression as

$$\mathsf{ct}'(\mathsf{sk}) = P^{-1} \cdot \sum_{j=0}^{n_{\mathrm{g}}} \phi_{n_{\mathrm{b}} j}(\tilde{b}_j + \tilde{a}_j \cdot \mathsf{sk}) + \varepsilon'_5 = P^{-1} \cdot \sum_{j=0}^{n_{\mathrm{g}}} \phi_{n_{\mathrm{b}} j}(\tilde{\mathsf{ct}}'_j(\mathsf{sk})) + \varepsilon'_5,$$

in which $\varepsilon'_5 := \sum_{j=0}^{n_{\mathrm{g}}-1} \phi_{n_{\mathrm{b}} j}(\varepsilon_4 \cdot \mathsf{sk}) + \varepsilon_5$ satisfies $\|\varepsilon'_5\|_\infty \le n_{\mathrm{g}} \|\varepsilon_4\|_\infty \|\mathsf{sk}\|_1 + \|\varepsilon_5\|_\infty$. Now apply (17) to obtain

$$\mathsf{ct}'(\mathsf{sk}) = P^{-1} \sum_{j=0}^{n_{\mathrm{g}}} \phi_{n_{\mathrm{b}} j} \left(P \sum_{i=0}^{n_{\mathrm{g}}} \phi_{-n_{\mathrm{b}} j}(\tilde{m}_{n_{\mathrm{b}} j + i}) \cdot \phi_i(\mathsf{m}) + \varepsilon_2 \right) + \varepsilon'_5.$$

Since the automorphisms distribute over addition, this decryption is equal to the sum of the desired noiseless result $\sum_{k=0}^{n-1} \tilde{m}_k \cdot \phi_k(\mathsf{m})$ plus an error term ε' such that

$$\|\varepsilon'\|_\infty \leq P^{-1} n_{\mathrm{g}} \|\varepsilon_2\|_\infty + \|\varepsilon'_5\|_\infty. \tag{18}$$

Substitute all the error bounds into (18), eliminate the term $P^{-1} \cdot C$ by using the assumption $P > C$, and add the factor q_l^{-1} and the noise term B_{rs} introduced by the final rescaling step to establish the stated inequality (16). □

It is reassuring to know that the dominant term in the upper bound (16) indeed comes from the n plaintext–ciphertext multiplications. The fact that the other part of the bound depends on n_{g} but not on n_{b} also deserves mention, because it suggests that taking n_{b} to be larger than n_{g} is advantageous not only from the standpoint of speed but also from that of noise growth.

6 Conclusion

We have constructed a general and flexible procedure that performs the key-switch of multiple polynomials in batch, while incorporating the gadget decomposition technique. This has enabled us to design an optimized multiplication circuit that accepts multiple ciphertext inputs of arbitrary degree in the secret-key indeterminate S, while reducing the number of modulus lowerings required.

This operation mode imposes increased memory requirements with respect to traditional non-batched approaches, but emerging memory architectures can alleviate the increased memory footprint and data movement pressure. Batched multiplication may be used in combination with other types of homomorphic operations. For example, we have shown that it can form part of homomorphic circuits implementing certain nonlinear polynomial approximations, improving performance when integrated judiciously. Further improvements in the construction may include directly applying the optimizations for rescaling suggested by Akherati *et al.* [2], which decrease the number of NTT operations when several multiplications are performed in a row before the key-switching; it may also be directly extended with the double-decomposition method of Kim *et al.* [23].

In addition, we introduced a method to generate multiple rotations of a ciphertext efficiently in batch, and employ its noise growth bounds in order to analyze the double-hoisted BSGS matrix–vector multiplication algorithm. Our results provide noise-based guidance for the selection of the baby and giant step parameters, complementing existing efficiency-based criteria in the literature.

References

1. Akherati, S., Tang, Y.J., Zhang, X.: Three-input ciphertext multiplication for homomorphic encryption. In: IEEE International Symposium on Circuits and Systems (ISCAS). pp. 1–5 (2025)
2. Akherati, S., Zhang, X.: Multi-input ciphertext multiplication for homomorphic encryption. arXiv preprint arXiv:2601.15401 (2026)

3. Al Badawi, A., Bates, J., Bergamaschi, F., Cousins, D.B., Erabelli, S., Genise, N., Halevi, S., Hunt, H., et al.: OpenFHE: Open-source fully homomorphic encryption library. In: 10th Workshop on Encrypted Computing & Applied Homomorphic Cryptography (2022)
4. Ali, A., Choi, J., Gipson, B., Gorantala, S., Kun, J., Legiest, W., Lim, L., Viand, A., Demissie, M.Z., Zheng, H.: HEIR: A universal compiler for homomorphic encryption (2025), https://arxiv.org/abs/2508.11095
5. Bajard, J.C., Eynard, J., Hasan, M.A., Zucca, V.: A full RNS variant of FV like somewhat homomorphic encryption schemes. pp. 423–442 (2016). https://doi.org/10.1007/978-3-319-69453-5_23
6. Barik, T., Mehta, P., Pindado, Z., Gupta, H., Kabra, M., Sadrosadati, M., Mutlu, O., Peña, A.J.: Long integer NTT execution on UPMEM-PIM for 128-bit secure fully homomorphic encryption. Future Generation Computer Systems **180**, 108386 (2026). https://doi.org/https://doi.org/10.1016/j.future.2026.108386, https://www.sciencedirect.com/science/article/pii/S0167739X26000208
7. Bossuat, J.P., Mouchet, C., Troncoso-Pastoriza, J.R., Hubaux, J.P.: Efficient bootstrapping for approximate homomorphic encryption with non-sparse keys. pp. 587–617 (2021). https://doi.org/10.1007/978-3-030-77870-5_21
8. Cheon, J.H., Han, K., Kim, A., Kim, M., Song, Y.: Bootstrapping for approximate homomorphic encryption. pp. 360–384 (2018). https://doi.org/10.1007/978-3-319-78381-9_14
9. Cheon, J.H., Han, K., Kim, A., Kim, M., Song, Y.: A full RNS variant of approximate homomorphic encryption. pp. 347–368 (2019). https://doi.org/10.1007/978-3-030-10970-7_16
10. Cheon, J.H., Kim, A., Kim, M., Song, Y.S.: Homomorphic encryption for arithmetic of approximate numbers. pp. 409–437 (2017). https://doi.org/10.1007/978-3-319-70694-8_15
11. Cheon, J.H., Kim, D., Kim, D., Lee, H.H., Lee, K.: Numerical method for comparison on homomorphically encrypted numbers. Cryptology ePrint Archive, Report 2019/417 (2019), https://eprint.iacr.org/2019/417
12. Damgård, I., Pastro, V., Smart, N.P., Zakarias, S.: Multiparty computation from somewhat homomorphic encryption. pp. 643–662 (2012). https://doi.org/10.1007/978-3-642-32009-5_38
13. Dathathri, R., Kostova, B., Saarikivi, O., Dai, W., Laine, K., Musuvathi, M.: EVA: An encrypted vector arithmetic language and compiler for efficient homomorphic computation. In: ACM SIGPLAN Conf. on Programming Language Design and Implementation (2020)
14. Ebel, A., Garimella, K., Reagen, B.: Orion: A fully homomorphic encryption framework for deep learning. In: Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), Volume 2. pp. 734–749 (2025). https://doi.org/10.1145/3676641.3716008, https://doi.org/10.1145/3676641.3716008
15. EPFL-LDS, Tune Insight SA: Lattigo v6 (Aug 2024), https://github.com/tuneinsight/lattigo
16. Genise, N., Micciancio, D., Polyakov, Y.: Building an efficient lattice gadget toolkit: Subgaussian sampling and more. pp. 655–684 (2019). https://doi.org/10.1007/978-3-030-17656-3_23
17. Gentry, C.: A fully homomorphic encryption scheme. Ph.D. thesis, Stanford University (2009), crypto.stanford.edu/craig

18. Goldschmidt, R.E.: Applications of division by convergence. In: Thesis (M.S.)–Massachusetts Institute of Technology, Dept. of Electrical Eng. (1964), https://api.semanticscholar.org/CorpusID:109445587
19. Halevi, S., Shoup, V.: Algorithms in HElib. In: Advances in Cryptology (CRYPTO). pp. 554–571 (2014)
20. Halevi, S., Shoup, V.: Faster homomorphic linear transformations in HElib. In: Advances in Cryptology (CRYPTO). pp. 93–120 (2018)
21. Han, K., Ki, D.: Better bootstrapping for approximate homomorphic encryption. In: Topics in Cryptology—CT-RSA. pp. 364–390 (2020)
22. Karatsuba, A.A.: The complexity of computations. Proceedings of the Steklov Institute of Mathematics-Interperiodica Translation **211**, 169–183 (1995)
23. Kim, M., Lee, D., Seo, J., Song, Y.: Accelerating HE operations from key decomposition technique. In: Advances in Cryptology (CRYPTO). pp. 70–92 (2023)
24. Lee, J.W., Kang, H., Lee, Y., Choi, W., Eom, J., Deryabin, M., Lee, E., Lee, J., Yoo, D., Kim, Y.S., et al.: Privacy-preserving machine learning with fully homomorphic encryption for deep neural network. IEEE Access **10** (2022)
25. Lloret-Talavera, G., Jorda, M., Servat, H., Boemer, F., Chauhan, C., Tomishima, S., Shah, N.N., Peña, A.J.: Enabling homomorphically encrypted inference for large DNN models. IEEE Transactions on Computers **71**(5), 1145–1155 (2021)
26. Lu, W., Huang, Z., Gu, Z., Li, J., Liu, J., Hong, C., Ren, K., Wei, T., Chen, W.: BumbleBee: Secure two-party inference framework for large transformers. In: ISOC Network and Distributed System Security Symposium (NDSS) (2025)
27. Lyubashevsky, V., Peikert, C., Regev, O.: On ideal lattices and learning with errors over rings. pp. 1–23 (2010). https://doi.org/10.1007/978-3-642-13190-5_1
28. Moon, J., Yoo, D., Jiang, X., Kim, M.: THOR: Secure transformer inference with homomorphic encryption. In: Proceedings of the ACM SIGSAC Conference on Computer and Communications Security. pp. 3765–3779 (2025)
29. Park, S., Song, W., Nam, S., Kim, H., Shin, J., Lee, J.: HEaaN.MLIR: An optimizing compiler for fast ring-based homomorphic encryption. Proc. ACM Program. Lang. **7**(PLDI) (2023). https://doi.org/10.1145/3591228, https://doi.org/10.1145/3591228
30. Pindado, Z., Spendlhofer, T., Allam, M., Mehta, P., Martens, L., Peña, A.J.: SoK of private deep neural network inference with approximate fully homomorphic encryption. Cryptology ePrint Archive, Paper 2026/047 (2026), https://eprint.iacr.org/2026/047
31. Regev, O.: On lattices, learning with errors, random linear codes, and cryptography. Journal of the ACM (JACM) **56**(6), 1–40 (2009)
32. Microsoft SEAL (release 4.3). https://github.com/Microsoft/SEAL (Apr 2026), microsoft Research, Redmond, WA.
33. Stehlé, D., Steinfeld, R., Tanaka, K., Xagawa, K.: Efficient public key encryption based on ideal lattices. In: Advances in Cryptology (ASIACRYPT). pp. 617–635 (2009)
34. Zama: Concrete: TFHE compiler that converts Python programs into FHE equivalent (2022), https://github.com/zama-ai/concrete
35. Zhang, J., Yang, X., He, L., Chen, K., Lu, W.j., Wang, Y., Hou, X., Liu, J., Ren, K., Yang, X.: Secure transformer inference made non-interactive. In: The Network and Distributed System Security (NDSS) Symposium (2025)